UNIVERSITATEA BABEȘ-BOLYAI

Facultatea de Științe Economice și Gestiunea Afacerilor

Informatică Economică

Lucrare de licență

Coordonator științific,

Lect. Univ. dr. Florina **Covaci**

2025

UNIVERSITATEA BABEȘ-BOLYAI

Facultatea de Științe Economice și Gestiunea Afacerilor

Informatică Economică

Lucrare de licență

Box Configurator – aplicație web pentru configurarea automată a ambalajelor din carton

Coordonator științific,

Lect. Univ. dr. Florina **Covaci**

2025

Rezumat

Lucrarea de față propune dezvoltarea unei aplicații web destinată optimizării procesului de comandă pentru ambalaje din carton. Aplicația permite clienților să încarce o fotografie cu obiectul pentru care au nevoie de ambalaj, iar prin intermediul unui configurator, sistemul estimează automat dimensiunile necesare ale cutiei. Utilizatorii pot introduce și manual aceste dimensiuni, precum și numărul de cutii dorite. Pe baza acestor date, aplicația calculează modelul de carton adecvat, dimensiunile optime ale coalei necesare și minimizează risipa de material. Comanda este transmisă către firmă într-un format standardizat, iar sistemul generează automat proiectul tehnic pentru tăiere CNC. Această soluție aduce beneficii semnificative atât clienților, cât și firmelor producătoare, oferind un proces rapid, eficient și digitalizat. În contextul în care astfel de aplicații lipsesc în acest domeniu, proiectul propus se remarcă prin caracterul său practic și inovator.

Cuprins

Rezumat.....

Listă Figuri și Tabele.....

Introducere 1

Raport de Analiză 2

1. **Identificarea și Descrierea Problemei** 3

1.1. Motivație 5

1.2. Context 6

1.2.1. Fațetă subiect 6

1.2.2. Fațetă utilizare 7

1.2.3. Fațetă IT 8

1.2.4. Fațetă Dezvoltare 9

2. **Cerințe sistem** 10

2.1. Surse de cerințe 10

2.2. Eliciția cerințelor 11

2.2.1. Observația directă 11

2.2.2. Interviuri 11

2.2.3. Brainstorming 11

2.3. Documentarea cerințelor 12

3. **Model de dezvoltare** 15

3.1. Justificarea modelului de dezvoltare 16

3.2. Etapele dezvoltării aplicației 17

3.3. Concluzii privind modelul de dezvoltare 18

Proiectarea sistemului informatic 18

4. **Proiectarea logică** 20

4.1. Arhitectura sistemului informatic 21

5. **Proiectarea Tehnică** 22

5.1. Structura generală 23

5.2. Structura fizică a datelor 24

5.3. Fluxurile de date 26

5.3.1. Estimarea automată 26

5.3.2. Trimiterea și salarea comenzii 27

5.4. Tehnologii specifice 28

6. Implementarea aplicației 29
6.1. Configuratorul de cutie 29
6.1.1. Determinarea suprafeței desfășurate 30
6.1.2. Aplicarea tarifului în funcție de grosime 31
6.1.3. Calculul costului total și unitar 31
6.2. Integrarea API-ului extern OpenAI 32
6.3. Algoritm de optimizare a cosnumului de materie primă 33
6.3.1. Desfășurarea dimensiunilor 35
6.3.2. Determinarea flapei de lipire 35
6.3.3. Calculul clapetelor de acoperire 36
6.3.4. Definirea suprafeței necesare pentru tăiere 37
6.3.5. Iterarea prin toate dimensiunile standard de coală 37
6.4. Generarea proiect tăiere pentru producție 40
7. Testarea aplicației 42
Concluzii și direcții viitoare 47
Bibliografie 52

Listă Figuri și Tabele

Figura 1: Diagrama Fishbone 5
Figura 2: Modelul Use-case 15
Figura 3: Modelul incremental 18
Figura 4: Tabele SQL 21
Figura 5: Structură multi-layer 22
Figura 6: Structura proiectului în mediul de lucru 24
Figura 7: Tabele SQL 26
Figura 8: Calcul 1 31
Figura 9: Calcul 2 31
Figura 10: Calcul 3 31
Figura 11: Calcul 4 32
Figura 12: Calcul 5 32
Figura 13: Standard FEFCO 201 34
Figura 14: Calcul 6 35
Figura 15: Calcul 7 36
Figura 16: Calcul 8 36
Figura 17: Calcul 9 37
Figura 18: Calcul 10 37
Figura 19: Calcul 11 38
Figura 20: Calcul 12 38
Figura 21: Calcul 13 39
Figura 22: Calcul 14 39
Figura 23: Calcul 15 39
Figura 24: Calcul 16 40
Figura 25: Calcul 17 40
Figura 26: Calcul 18 41
Figura 27: Calcul 19 42
Figura 28: Autentificare 44
Figura 29: Autentificare 2 45
Figura 30: Detalii comandă 45
Figura 31: Verificarea integrității schiței 46

Tabel 1: Roluri și obiective 12
Tabel 2: Cazuri de utilizare 13
Tabel 3: Sarcini de testare 44

Introducere

Pentru majoritatea produselor de pe piață, industria ambalajelor din carton reprezintă un element crucial în lanțul de logistică și distribuție. Deoarece este reciclabil, ușor și rezistent, ambalajul din carton este preferat în practic fiecare sector comercial, inclusiv industria alimentară, electronica, cosmeticele și bunurile de uz casnic. Companiile din sector sunt obligate să se adapteze la așteptările din ce în ce mai variate ale clienților, care caută soluții de fabricație a ambalajelor rapide, personalizate și automatizate, într-o eră în care sustenabilitatea și eficiența proceselor sunt obiective globale. Industria ambalajelor continuă să evolueze spre soluții mai eficiente și sustenabile, conform tendințelor globale observate (Soroka, 2002).

Cu toate acestea, în ciuda semnificației sectorului, majoritatea procedurilor de configurare și comandare a ambalajelor personalizate sunt încă realizate în mod tradițional, prin apeluri telefonice, e-mailuri sau consultații față în față. Acest lucru necesită o cantitate semnificativă de timp și bani atât din partea clientului, cât și a companiei de producție. În plus, mulți clienți nu cunosc măsurătorile precise ale obiectului pentru care au nevoie de o cutie sau nu știu cum să comunice corect aceste informații companiei de ambalare. Acest lucru poate duce la erori, întârzieri sau articole neconforme.

Această lucrare propune crearea unei aplicații web de ultimă generație care să simplifice și să automatizeze procesul de comandă pentru ambalajele din carton, în scopul de a răspunde acestor cerințe actuale ale pieței.

Cu ajutorul unui configurator ușor de utilizat, aplicația dezvoltată permite utilizatorului să trimită o fotografie a obiectului pentru care dorește o cutie, iar sistemul determină automat dimensiunile necesare ale ambalajului. Utilizatorul poate alege câte cutii dorește, iar programul va gestiona toate aceste date înainte de a trimite comanda într-un format standard către afacere. Incorporarea unui algoritm care determină automat tipul de carton necesar, dimensiunea ideală a foii și cantitatea de deșeuri de material este o caracteristică nouă a aplicației. Partea de producție a afacerii beneficiază enorm de această caracteristică, deoarece automatizează procedurile de optimizare și calcul, reducând timpul de execuție și risipa de material. În plus, aplicația pregătește automat comanda de tăiere prin crearea unui proiect tehnic care este compatibil cu mașinile CNC (cum ar fi Zünd).

Deoarece nu există alternative disponibile pe piață în acest moment care să combine toate aceste caracteristici într-o singură platformă web, aplicația noastră are un caracter unic și util. Ajută clienții care nu au expertiză tehnică sau echipamente de măsurare precise, precum și

producătorii de ambalaje care pot automatiza și accelera procesul de preluare și procesare a comenzilor. Designul aplicației subliniază ușurința procesului de configurare și oferă utilizatorilor o experiență prietenoasă și intuitivă, pe lângă caracteristicile sale fundamentale. Un spectru larg de clienți poate acum beneficia de un serviciu modern și eficient datorită designului ușor de utilizat al interfeței web, chiar și pentru cei fără expertiză tehnică. În plus, crearea aplicației ia în considerare cerințele reale ale companiilor de ambalare, care se confruntă frecvent cu cerințe inexacte sau incomplete din partea clienților, ceea ce duce la întârzieri, refaceri și pierderi de materiale. Instrumentul ajută la simplificarea procesului de producție și la minimizarea erorilor umane prin standardizarea datelor primite și crearea automată a fișierelor tehnice necesare. Această lucrare de licență își propune să demonstreze valoarea combinării automatizării și tehnologiilor web în domeniile industriale convenționale, oferind în același timp o soluție viabilă. Această strategie deschide noi perspective pentru digitalizarea proceselor de producție și pentru creșterea competitivității afacerilor din sectorul ambalajelor.

În concluzie, efortul planificat vizează atât îmbunătățirea experienței utilizatorului final în ceea ce privește furnizorii de ambalaje, cât și digitizarea și optimizarea operațiunilor companiilor din această industrie. Acest program, care combină tehnologii contemporane precum configurarea dinamică, estimarea dimensiunilor, încărcarea imaginilor și calculul optimizat al materialelor, reprezintă un pas semnificativ către digitalizarea sectorului de ambalaje personalizate din carton.

Raport de Analiză

Scopul acestui capitol este de a prezenta o imagine de ansamblu a circumstanțelor care au dus la identificarea nevoii de a dezvolta o aplicație web pentru plasarea comenzilor de ambalaje din carton, precum și motivele care au dus la efortul discutat aici. Vom face o analiză detaliată a problemei specifice pe care aplicația o abordează, precum și a motivațiilor personale și practice care au dus la crearea proiectului, precum și a cadrului tehnologic și economic în care această teză este aplicabilă.

În această secțiune, v-or fi explicate elementele care au dus la selecția problemei, precum și modul în care aceasta răspunde cerințelor actuale ale industriei și particularităților domeniului vizat. Acest capitol conține un studiu al procedurilor care sunt în prezent utilizate de companiile de ambalare. Se concentrează pe obstacolele pe care le întâmpină clienții atunci când vine vorba de comunicarea comenzilor lor, precum și pe dificultățile tehnologice cu care

se confruntă afacerile atunci când nu au acces la un instrument digital pentru configurarea și optimizarea pachetelor.

Există trei subsecțiuni esențiale care vor fi prezentate în acest raport: identificarea și descrierea problemei, care va discuta problema principală ce justifică necesitatea aplicației; motivația, care va detalia raționamentul personal și profesional al autorului pentru alegerea acestui subiect; și contextul, care va descrie modul în care aplicația este integrată într-un cadru tehnologic și de afaceri relevant. În etapele ulterioare ale lucrării, cum ar fi proiectarea, dezvoltarea și testarea aplicației, aceste componente vor servi drept bază pe care se vor construi etapele ulterioare.

1. Identificarea și Descrierea Problemei

Piața ambalajelor din carton a înregistrat o expansiune globală substanțială în ultimii ani, în principal ca urmare a dezvoltării rapide a comerțului electronic și a cererii tot mai mari pentru opțiuni de ambalare ecologice. Conform unui sondaj publicat nu cu mult timp în urmă, valoarea acestei piețe este estimată să ajungă la 272,18 miliarde USD până în anul 2034, cu o rată anuală compusă de creștere (CAGR) de 4% în perioada analizată. În 2025, se preconizează că piața va valora 191,23 miliarde USD. Aceste date oferă dovezi suplimentare că industria ambalajelor este de importanță strategică, în special în ceea ce privește capacitatea sa de a se adapta cerințelor pieței și tendinței comportamentului consumatorilor, care se concentrează din ce în ce mai mult pe eficiență și personalizare.

Necesitatea automatizării în procesele de producție și livrare a ambalajelor este adusă în atenția unui număr tot mai mare de studii și evaluări de piață, care are loc simultan cu această creștere. Companiile care adoptă soluții automatizate în procesele lor interne pot realiza economii de până la 25% din costurile de conversie a produselor (Boston Consulting Group, 2012). Aceste economii pot fi realizate și în industria ambalajelor. Utilizarea automatizării duce la o serie de beneficii directe, inclusiv reducerea erorilor umane, reducerea deșeurilor de materiale, scurtarea timpilor de execuție și, implicit, îmbunătățirea satisfacerii nevoilor clienților.

În același timp, soluțiile de ambalare la cerere, cum ar fi cele dezvoltate de Packsize, oferă exemple clare de bune practici: prin generarea automată a cutiilor pe baza dimensiunilor reale ale produselor, stocul inutil este eliminat, logistica este optimizată, iar volumul de deșeuri rezultat din tăieturi ineficiente este redus semnificativ.

În lumina acestui fapt, crearea unei aplicații web care să permită consumatorilor să plaseze comenzi de ambalaje din carton într-un mod prietenos și automatizat, fie prin introducerea dimensiunilor unui obiect, fie prin încărcarea unei imagini care poate fi interpretată automat, devine o necesitate urgentă în prezent. Având un astfel de sistem, se oferă beneficii evidente atât clienților, care pot comanda ambalajul necesar într-un mod rapid și corect, cât și producătorilor de ambalaje, care pot gestiona comenzile într-un mod eficient și standardizat.

Un sistem digital care se concentrează pe minimizarea erorilor, creșterea vitezei de procesare și optimizarea utilizării materialelor în sectorul ambalajelor din carton este subiectul acestui articol, care încearcă să analizeze, să proiecteze și să execute un astfel de sistem.

În plus față de aceste provocări operaționale, există și o lipsă de deschidere în comunicarea care are loc între companie și client. Momentan, clientul nu are acces în timp real la progresul comenzii sau confirmarea că datele introduse sunt corecte și ar trebui să fie procesate corect. Absența unui sistem digital care centralizează comenzile, validează automat dimensiunile și oferă o previzualizare a ambalajului final nu doar că crește incertitudinea, dar diminuează și încrederea clientului în serviciul oferit. Aceste deficiențe devin dezavantaje competitive semnificative într-o piață extrem de competitivă, când termenele de livrare și personalizarea rapidă sunt de cea mai mare importanță.

Astfel, este necesar să găsim o soluție care nu doar automatizează procesul, ci oferă și un mediu de angajament digital care să fie atât profesional, cât și ușor de înțeles. Acest lucru va duce la un câștig atât în eficiența operațională, cât și în satisfacția clienților.

Concluzionând, problema în discuție are valoare dublă: pe de o parte, dificultatea pe care clientul o are în a comanda ambalajul corespunzător, iar pe de altă parte, dificultatea pe care compania o are în procesarea comenzilor într-un mod eficient. Consecințele acestor probleme includ pierderi de timp, bani și oportunități valoroase. O aplicație care digitalizează, automatizează și optimizează această procedură este propusă în acest studiu. Aplicația ar aduce beneficii majore ambelor părți implicate în proces.

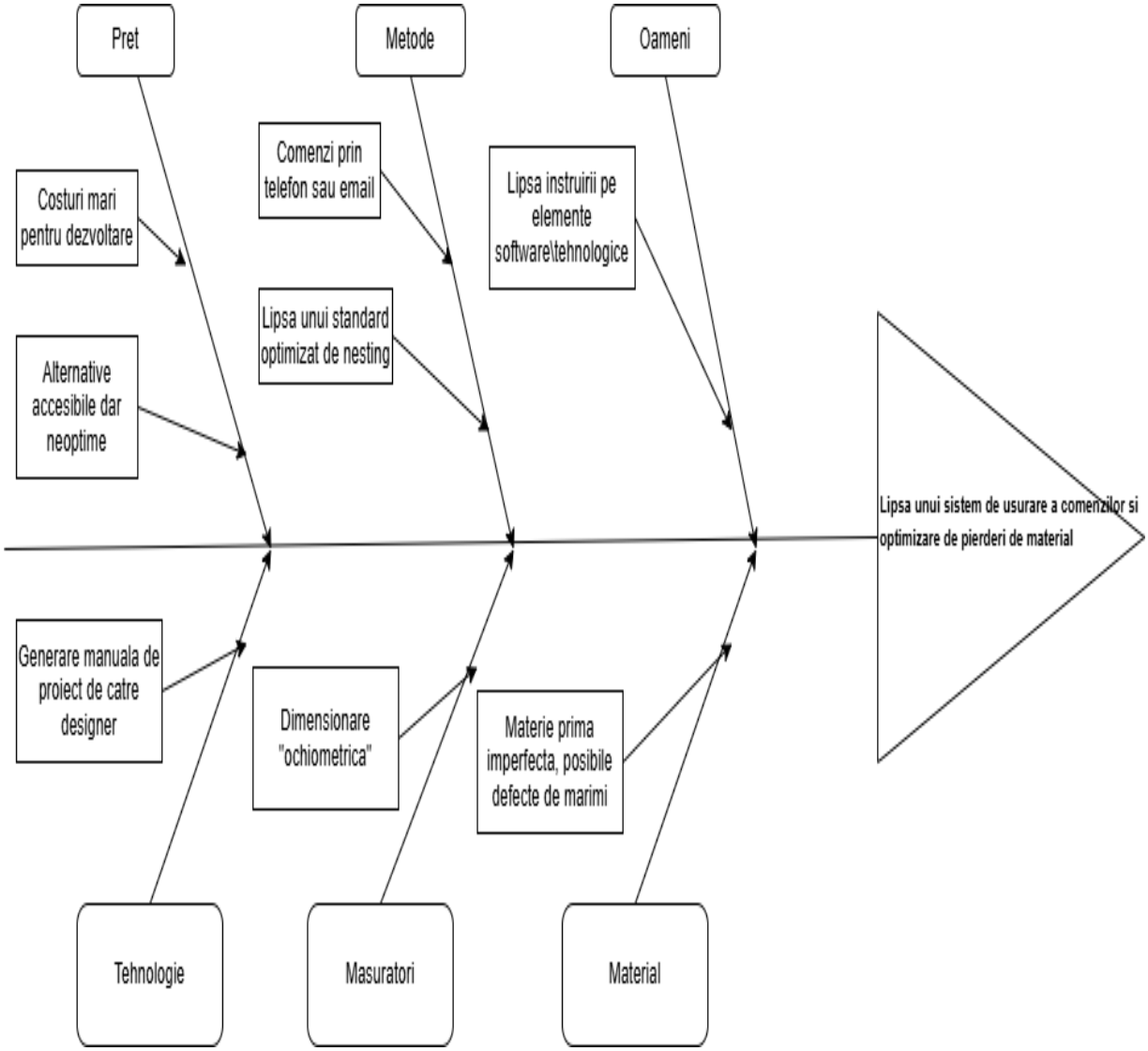


Figura 1: Diagrama Fishbone

1.1. Motivație

Motivul principal pentru dezvoltarea acestei aplicații este identificarea unei nevoi reale de piață, precum și aspirația de a oferi o soluție tehnică care să fie atât practică, cât și imediat utilizabilă. După ce am observat modul laborios în care clienții interacționează cu companiile care fabrică ambalaje, am realizat că digitalizarea acestui proces ar aduce o serie de oportunități de îmbunătățire. Clienții ar beneficia de o metodă simplă, actualizată și rapidă pentru a obține ambalajul personalizat, în timp ce companiile ar economisi timp, ar reduce numărul de erori cauzate de intervenția umană și ar maximiza cantitatea de carton consumată.

În plus, experiențele concrete obținute din conversațiile cu producătorii locali de ambalaje au scos la iveală în ce măsură munca angajaților ar fi ușurată dacă li s-ar oferi un instrument care să calculeze automat dimensiunile cutiilor, să optimizeze foaia de carton și să furnizeze fișierul gata de tăiat. Absența unei astfel de aplicații, împreună cu cantitatea substanțială de muncă manuală, constituie o dificultate cu care aceste afaceri se confruntă foarte frecvent.

În același timp, ca student în dezvoltarea aplicațiilor web și interesat de încorporarea soluțiilor tehnice în operațiunile de afaceri reale, am recunoscut în această problemă oportunitatea ideală de a combina cunoștințele academice cu aplicarea practică. Dezvoltarea unei platforme care utilizează tehnologii contemporane, cum ar fi detectarea obiectelor în fotografii, configurarea automată, algoritmi de optimizare și crearea de fișiere CNC, oferă un cadru adecvat pentru aplicarea cunoștințelor acumulate și aduce o contribuție tangibilă industriei.

În plus față de a fi un proiect academic, acest studiu este și un pas către o soluție practică care poate fi modificată, dezvoltată și implementată în lumea reală.

1.2. Context

În această secțiune este prezentat contextul general în care a fost dezvoltată aplicația web pentru automatizarea comenzilor de ambalaje din carton. Lucrarea se bazează pe o nevoie reală observată în piață, unde lipsa digitalizării procesului de configurare și comandă a cutiilor personalizate afectează negativ eficiența firmelor și experiența clientului. Contextul va fi descris utilizând metodologia fațetelor, care acoperă patru dimensiuni esențiale: subiect, utilizare, IT și dezvoltare.

1.2.1. Fațeta subiect

Aplicația Kaydara Box Configurator își propune să modernizeze un proces consacrat, dar încă preponderent segmentat, din industria ambalajelor de carton. În prezent, majoritatea atelierelor din România, inclusiv Kaydara, primesc comenzile sub forma unor fișiere Excel sau e-mailuri care conțin doar cele patru măsurători esențiale: lungime, lățime, înălțime și grosime a cartonului, apoi trec printr-o serie de etape manuale: transcrieri pe hârtie, calcule cu ruleta, schițe în Adobe Illustrator și probe fizice de tăiere. În acest lanț de operațiuni, deciziile de alegere a colilor se bazează pe intuiție și experiență, nu pe date concrete, ceea ce conduce la pierderi de carton de până la 25% și întârzieri de livrare.

Pe de o parte, configuratorul nostru este primul de acest fel disponibil pe piața din România, oferind clientului posibilitatea de a introduce dimensiunile direct sau de a încărca o fotografie pentru estimare automată printr-un serviciu AI. Această abordare îndrăzneață elimină complet etapa preliminară de măsurare manuală și reduce considerabil riscul erorilor umane. Utilizatorii pot astfel să obțină în câteva secunde nu doar un cost estimativ, dar și un fișier SVG pregătit pentru mașina de tăiere, fără a apela la un designer grafic și fără a parcurge ciclurile interminabile de revizuire în Illustrator.

Pe de altă parte, motorul de optimizare înlocuiește vechea metodă a „alegerii instinctive" a unei coli de carton cu un algoritm ce calculează exact modul în care multiple desfășurate de cutii pot fi aliniate pe coli standard pentru a minimiza deșeurile. În timp ce acum operatorii fac decizii uneori eronate pe baza experienței și fac calcule aproximative pe hârtie, sistemul tehnic determină rapid și riguros configurația ideală, reducând risipa și nevoia de retușuri. Astfel, subiectul proiectului îmbină două inovații: introducerea unui configurator inteligent, susținut de AI, și implementarea unui nesting automat care asigură utilizarea rezonabilă și sustenabilă a materialului.

1.2.2. Fațeta utilizare

Kaydara Box Configurator este gândit ca un sistem software cu două fețe complementare: una orientată către client, care configurează și plasează comanda de cutii, și alta destinată echipei de business, care primește plus-valoare prin tabloul de optimizare și fișierele de producție. În centrul experienței stă un design aerisit, cu meniuri clare: „Despre", „Avantaje" şi „Configurator", care îi ghidează pe utilizatori în fiecare pas fără să îi copleșească cu opțiuni tehnice.

Primul pas este autentificarea, recomandată pentru toate sesiunile, iar dacă un utilizator are rol de administrator, în bara de navigare îi apare suplimentar secțiunea „Business". Aici, administratorul are acces nerestricționat: vizualizează lista tuturor comenzilor, poate vedea imaginea încărcată de client, descarcă fișierul SVG și analizează în detaliu coloanele de optimizare, precum coala aleasă, numărul de coli necesare, procentul de risipă și cutii/coală. În acest mod, echipa de producție preia rapid fișierele gata de productie și are trasabilitate completă asupra fiecărei comenzi.

Pentru utilizatorii obișnuiți, viitorii clienți Kaydara, secțiunile „Despre" şi „Avantaje" oferă contextul serviciului, iar configuratorul este locul în care magia se întâmplă. În formularul de configurare, clientul poate completa manual dimensiunile L × l × h sau, mai simplu, poate

încărca o fotografie a obiectului. Cu un singur clic pe „Estimează dimensiuni”, API-ul extrage valorile și completează automat câmpurile. După ce a introdus grosimea cartonului și numărul de bucăți dorite, clientul vede pe loc o fereastră în partea de jos cu prețul estimativ per comandă și per cutie. Dacă toate câmpurile sunt valide, butonul „Trimite comanda” devine activ și, odată apăsat, salvează automat datele în baza de date.

La finalul acestui proces intuitiv, clientului i s-a ușurat semnificativ procesul de comandă, iar administratorii își pot începe ziua cu un singur click în secțiunea Business, știind exact câte coli să pregătească și cum să optimizeze producția. Astfel, interfața rămâne simplă pentru client și puternică pentru echipa internă, îmbinând claritatea utilizării cu eficiența operațională.

1.2.3. Fațeta IT

Configuratorul Kaydara Box este bazat pe un stack tehnologic simplu, dar fiabil, care este ușor de găzduit și gestionat. Serverul rulează PHP 8.1 „vanilla”, fără framework-uri complicate, astfel încât toată logica de afaceri să fie accesibilă și ușor de înțeles.

Pentru autentificarea și administrarea utilizatorilor, sistemul folosește interogări parametrizate MySQL. Funcțiile integrate password_hash și password_verify criptează și verifică parolele. În timp ce sesiunile PHP mențin funcțiile de „client” sau „administrator”, restricțiile codului permit accesul la panoul de business și afișarea secțiunilor în mod corespunzător. Fiecare comandă trimisă de client este salvată automat în baza de date și in fisier CSV pe disc. Acest lucru permite un backup rapid și un format simplu de procesat folosind un editor de text de bază sau Excel.

Partea front-end este construită pe semantice HTML5 și CSS3 cu ajutorul kitului Bootstrap pentru un design curat și receptiv. Funcțiile importante sunt executate cu JavaScript pur: cererea AJAX către functia de estimare permite estimarea dimensiunilor imaginii, actualizarea prețului estimat în timp real și validarea numerică a câmpurilor. Pentru a asigura o experiență fluidă, am inclus un mic spinner CSS care apare atunci când serviciul AI validează imaginea și dispare atunci când primește datele de la server.

Două scripturi PHP diferite conțin logica specifică pentru optimizarea și generarea SVG: genereaza_svg.php creează dinamic un fișier XML SVG valid cu forme și etichete orientate corespunzător pentru mașinile CNC, iar optimize.php gestionează calculele geometrice, cum ar fi desfasurarea, suprafața totală și procentajul de carton neutilizat. Aceste

module păstrează logica de procesare și stratul de prezentare separat, returnând doar datele necesare pentru interfață și „linkul” către fișier, fără a divulga datele clientului. Fateta IT a proiectului acoperă fiecare aspect, de la server-side la client-side, prin utilizarea unor tehnologii bine cunoscute și ușor de controlat în orice context LAMP. Acest lucru garantează securitate, performanță și scalabilitate. Utilizarea CSV a fost o decizie bazata pe nevoia personalizata a firmei de compatibilitate cu diferite instrumente folosite de ei pentru alte scopuri. În plus, designul modular permite transferul în viitor la un cadru MVC (Laravel) sau externalizarea componentelor (microservicii de optimizare) fără a rescrie întregul cod.

1.2.4. Fațeta dezvoltare

Configuratorul Kaydara Box a fost dezvoltat folosind o metodă colaborativă, mai degrabă decât un pre-proiect strict. Prima dată, am lucrat câteva zile cu operatorii în atelier pentru a vedea cum procesează comenzile, inclusiv cum înregistrează dimensiunile, ce erori fac frecvent și cât timp pierd cu mostrele pe hârtie. Am vorbit cu managerul de producție, care și-a exprimat nemulțumirea față de cantitatea mare de carton care era irosită, și cu designerul grafic, care introducea manual măsurătorile în Illustrator. După ce am făcut aceste observații, am construit o listă de bază cu „ce trebuie să facă aplicația” și am împărțit-o în bucăți gestionabile.

Am adaptat elemente din cadrul Scrum pentru organizarea proiectului de licență, creând un „mini-Scrum” personalizat. În fiecare săptămână stabileam două-trei obiective concrete, de exemplu: implementarea stocării comenzilor în CSV și afișarea costului estimat, integrarea funcționalității de estimare prin fotografie, apoi abordarea modulelor de administrare și autentificare. Pentru fiecare cerință, efectuăm teste intermediare (trimitere de comenzi multiple, încărcare de fotografii cu dimensiuni variate, completare a formularului de comandă cu date atipice) și verificăm consistența rezultatelor în baza de date și în fișierul CSV. Dacă identificam vreun bug, nu trecem mai departe până la remediere. Pentru a menține un istoric curat al codului, am folosit Git: înainte de a fuziona orice ramură în „main”, deschideam o ramură dedicată fiecărei funcționalități semnificative și solicitam coordonatorului să revizuiască codul. Astfel, am asigurat un flux de lucru structurat, cu verificări periodice și feedback constant.

Deși nu am implementat Scrum în forma sa completă (roluri dedicate de Product Owner și Scrum Master, ceremonii zilnice cu echipă, definiție rigidă a sprinturilor), am adoptat principiile principale: muncă iterativă, feedback rapid și capacitatea de a adapta prioritățile pe

parcurs. În planul pe termen lung, lista de cerințe (backlog-ul) rămâne deschisă constant, prioritățile pot fi modificate în funcție de necesități, iar fiecare etapă de dezvoltare avansează integrarea aplicației într-un magazin online real sau ca instrument dedicat unei societăți comerciale din sectorul de producție.

2. Cerințe de sistem

Acest capitol descrie procesul de identificare și structurare a cerințelor aplicației web pentru configurarea și comanda ambalajelor din carton. Cerințele de sistem reprezintă fundamentul dezvoltării aplicației, ele definind comportamentul așteptat, funcționalitățile necesare și constrângerile tehnice. Se vor prezenta sursele acestor cerințe, metodologia utilizată pentru elicitarea lor și modul în care au fost documentate. O înțelegere clară a cerințelor asigură un proces de dezvoltare coerent și adaptat nevoilor utilizatorilor finali și ale firmei producătoare.

2.1. Surse de cerințe

Cerințele sistemului propus au fost extrase din realitatea actuală a procesului de comandă a ambalajelor din carton. Lipsa unui sistem digital unitar care să permită clienților să configureze ușor ambalajele și să trimită rapid comenzi către firmă a dus la apariția acestei nevoi. Astfel, principalele surse de cerințe au fost:

- analiza fluxului tradițional de comandă din cadrul firmelor de ambalaje;
- feedback-ul primit de la clienți persoane fizice sau juridice, privind dificultatea alegerii cutiei potrivite;
- nevoia firmei de a eficientiza procesele interne de producție, prin automatizarea estimării materialelor și reducerea risipei;
- observația personală și experimentarea procesului de comandă în lipsa unui sistem automatizat;
- exemple similare din alte industrii, cum ar fi platformele de personalizare a produselor sau configuratoarele online de tip CAD.

2.2. Elicitația cerințelor

Pentru a colecta cerințele aplicației într-un mod riguros și orientat către nevoile reale ale utilizatorilor, au fost aplicate mai multe metode de elicitare. Cele mai relevante pentru acest proiect au fost observația directă, interviul și brainstorming-ul.

2.2.1. Observația directă

Observația a fost realizată în cadrul unei firme care produce ambalaje din carton, urmărind întregul proces de preluare a unei comenzi. Au fost identificate dificultăți semnificative în comunicarea cerințelor clientului prin telefon sau email. În multe cazuri, clienții nu furnizează dimensiuni precise sau nu le cunosc, ceea ce duce la multiple reveniri și confirmări. Personalul tehnic trebuie să interpreteze cerințele, să le transpună într-un model CAD și să estimeze manual coala necesară, adesea folosind metode empirice. Această observație a scos în evidență necesitatea unui sistem digital în care clientul să introducă datele într-un mod standardizat, iar firma să primească informațiile într-un format procesabil automat.

2.2.2. Interviuri

Au fost realizate interviuri semi-structurate cu patru clienți care au comandat recent cutii personalizate. Aceștia au menționat ca adaugarea unei opțiuni simple de comandat, fără canale precum email sau telefon, le-ar ușura procesul pentru unele tipuri de comenzi. De asemenea, doi operatori ai unei firme de producție au explicat cum procesele interne sunt încetinite de comenzile imprecise, iar lipsa unui sistem care să genereze automat un proiect de tăiere duce la pierderi materiale. Răspunsurile au fost centralizate și au stat la baza cerințelor de funcționalitate ale aplicației.

2.2.3. Brainstorming

Pentru a obține o imagine mai amplă asupra așteptărilor potențialilor utilizatori, a fost organizată o sesiune de brainstorming la care au participat studenți, dezvoltatori și persoane interesate de digitalizarea proceselor. Participanții au venit cu propuneri precum: integrarea unui estimativ de preț, optimizarea risipei de material, salvarea comenzilor anterioare sau integrarea cu aplicații mobile. Cele mai fezabile idei au fost selectate pentru a fi transpuse în funcționalitățile aplicației.

Următorul tabel prezintă principalele roluri și obiective ale actorilor implicați:

Tabel 1: Roluri și obiective

Actor	Roluri și obiective
Client	Comandă ambalaje prin încărcare imagine sau introducere dimensiuni; primește estimare și confirmare.
Operator firmă	Vizualizează comenzile; aprobă și organizează producția ambalajelor.
Sistem (backend)	Calculează automat coala necesară, optimizează risipa și generează proiectul pentru tăiere.

2.3. Documentarea cerințelor

Documentarea cerințelor s-a realizat într-un mod structurat și detaliat, împărțind cerințele identificate în două categorii principale:

Cerințe funcționale (CF): se referă la comportamentele explicite ale sistemului, la acțiunile pe care utilizatorii trebuie să le poată efectua și la modul în care sistemul răspunde acestor acțiuni.

Cerințe non-funcționale (CNF): specificații care definesc proprietățile calitative ale unui sistem software, fără a descrie funcționalitățile propriu-zise.

Pentru fiecare categorie s-a realizat o listă clară, care va sta la baza dezvoltării, testării și validării aplicației.

Cerințe funcționale (CF):

CF1: Utilizatorul trebuie să poată crea un cont și să se autentifice în platformă.

CF2: Aplicația trebuie să permită încărcarea unei imagini cu obiectul pentru care se dorește ambalajul.

CF3: Aplicația trebuie să estimeze automat dimensiunile obiectului pe baza imaginii încărcate.

CF4: Utilizatorul trebuie să aibă opțiunea de a introduce manual dimensiunile obiectului.

CF5: Sistemul trebuie să genereze automat proiectul de ambalaj și să estimeze coala necesară, cu minimizarea risipei.

CF6: Comanda trebuie să fie salvată în baza de date și transmisă către firma producătoare.

CF7: Firma trebuie să poată accesa comenzile, să le gestioneze și să le pregătească pentru producție.

Cerințe non-funcționale (CNF):

CNF1: Aplicația trebuie să fie compatibilă cu principalele browsere web (Chrome, Firefox, Edge).

CNF2: Timpul de răspuns al aplicației trebuie să fie sub 2 secunde pentru majoritatea acțiunilor.

CNF3: Interfața aplicației trebuie să fie intuitivă și ușor de utilizat de către utilizatori fără experiență tehnică.

CNF4: Sistemul trebuie să asigure integritatea și confidențialitatea datelor transmise și salvate.

CNF5: Platforma trebuie să fie scalabilă și să permită adăugarea ulterioară de noi funcționalități.

Această documentare a cerințelor oferă o bază solidă pentru implementarea și evaluarea ulterioară a aplicației. Fiecare cerință a fost formulată clar, concis și măsurabil, pentru a permite o urmărire eficientă în cadrul etapelor de dezvoltare și testare.

Modelul Use-Case

Tabel 2 : Cazuri de utilizare

Nr.	Caz de utilizare	Descriere
1	Autentificare client	Clientul se autentifică pentru a vizualiza comenzile anterioare și a plasa o comandă nouă.
2	Upload imagine obiect	Clientul încarcă o imagine pentru care se estimează automat dimensiunile cutiei.
3	Configurare manuală	Clientul introduce manual dimensiunile și numărul de cutii dorite.

4	Transmitere comandă	Datele configurate sunt trimise către firmă pentru procesare.
5	Optimizare coală carton	Aplicația determină dimensiunea optimă a coalei și minimizează risipa.

Din perspectiva utilizatorilor sau a sistemelor externe, un model de caz de utilizare oferă o abordare metodică pentru a explica toate caracteristicile pe care aplicația le oferă, precum și interacțiunile lor pentru a produce valoare. Pentru reprezentarea arhitecturii aplicației, s-a utilizat o diagramă UML de tip use-case, conform specificațiilor oferite de versiunea 2.5 a standardului UML publicat de Object Management Group (2015). În afară de un actor tehnic extern, API-ul terț pentru estimare de dimensiuni, avem trei actori cheie în diagrama "Magazin Online": Vizitatorul Ocazional, Utilizatorul Client și Utilizatorul Administrator. De la "Creează cont" și "Răsfoire" pentru vizitator, până la fluxurile de "Autentificare" și "Comandă cutii" pentru client, unde pasul "Plasează comandă" este marcat cu relația «include» ca fiind obligatorie – și până la "Gestionează comenzi," "Vizualizează tabelul comenzilor," "Descarcă proiect SVG," și "Primește informații despre optimizarea deșeurilor," fiecare caz de utilizare, reprezentat printr-o elipsă, arată un scenariu complet de interacțiune. Dacă adaugam cazuri opționale, «extend », evidențiază dependențele funcționale și scenariile alternative; limita sistemului definește clar ce rulează intern (stocarea datelor de autentificare și a istoricului comenzilor în MySQL) de ceea ce este externalizat (apeluri către OpenAI pentru estimarea dimensiunii). Proiectarea diagramelor de secvență, definirea specificațiilor API, construirea bazei de date și organizarea scenariilor de testare folosind acest model garantează că fiecare cerință funcțională este exprimată precis, poate fi urmărită și verificată până la nivelul codului și al utilizatorului final, oferind astfel un punct de plecare indispensabil pentru toate etapele următoare.

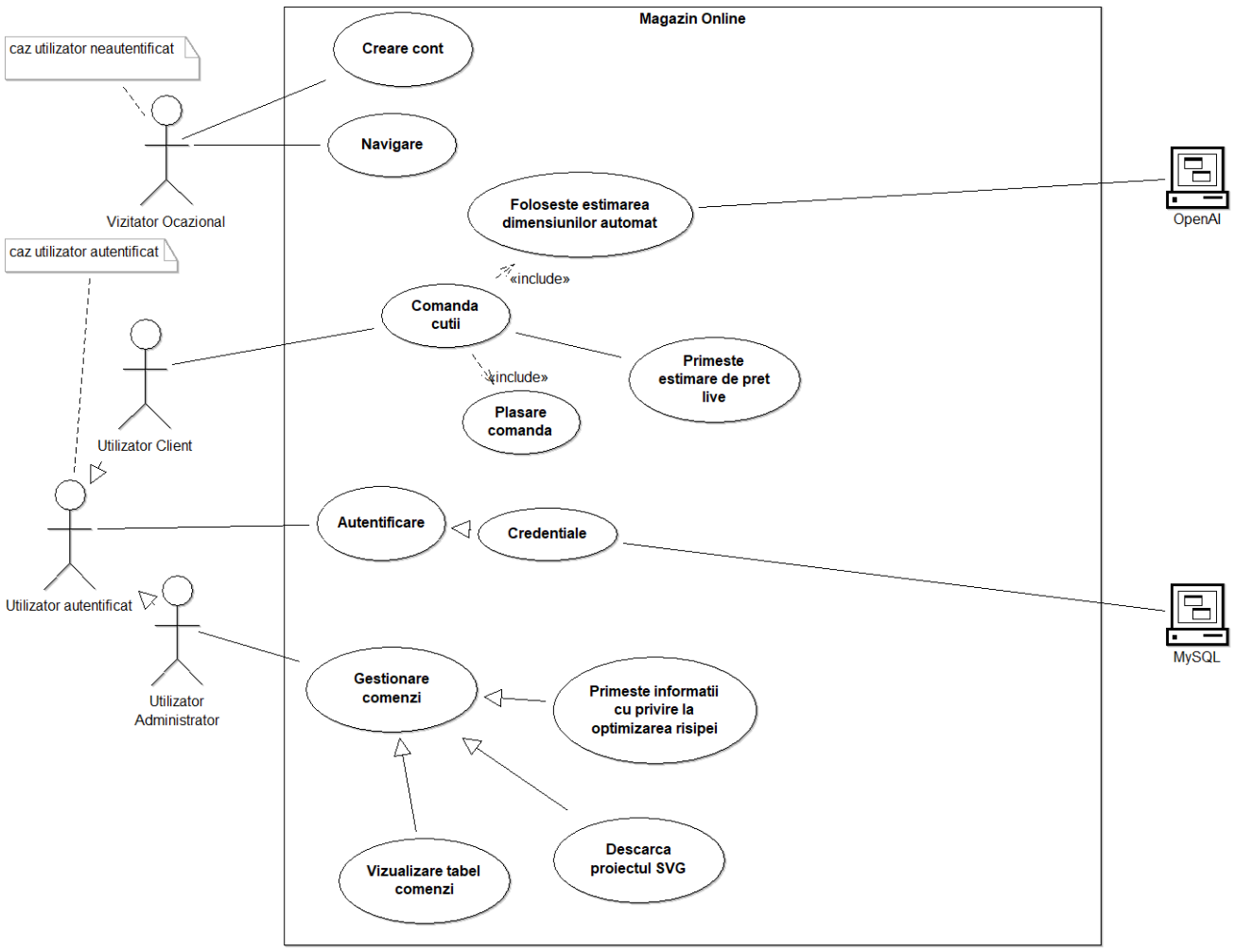


Figura 2: Modelul Use-case

3. Model de dezvoltare

Acest capitol are rolul de a descrie modelul de dezvoltare utilizat în cadrul aplicației web pentru configurarea și transmiterea comenzilor de ambalaje din carton. Alegerea unei metodologii potrivite de dezvoltare este esențială pentru succesul proiectului, deoarece

influențează modul de organizare a etapelor, nivelul de flexibilitate în integrarea feedback-ului și calitatea produsului final.

Pentru acest proiect, s-a adoptat un model iterativ-incremental, combinat cu practici inspirate din metodologia Agile. Alegerea acestui model s-a bazat pe caracteristicile aplicației: funcționalități independente, dar interconectate, complexitate medie și nevoia de testare continuă și de adaptare pe parcurs. Modelul incremental aplicat este unul potrivit pentru prototipuri web și reflectă bune practici în ingineria software (Sommervile, 2015).

Modelul incremental presupune dezvoltarea aplicației în pași succesivi (incrementări), fiecare dintre aceștia livrând o versiune funcțională a aplicației, care poate fi testată și validată. Practic, în fiecare etapă a fost adăugat un nou modul funcțional, testat individual, integrat și apoi validat împreună cu restul sistemului. Acest lucru a permis controlul asupra calității, posibilitatea de a integra rapid cerințe noi și o dezvoltare centrată pe utilizator.

3.1. Justificarea modelului de dezvoltare

Am folosit o abordare iterativ-incrementală pentru crearea aplicației de ambalare din carton, complet potrivită pentru munca mea particulară și pentru viteza specifică de redactare a lucrării de licență. În loc să încerc să descriu și să planific cu precizie fiecare etapă de la început, am împărțit proiectul în iterații de aproximativ două săptămâni. De la autentificare și încărcarea imaginilor până la calculul automat al dimensiunilor, optimizarea layout-ului ("nesting") și crearea fișierelor SVG pentru CNC, fiecare iterație a avut un obiectiv clar, concentrat pe o singură capacitate.

Obișnuiam sa enumar obiectivele minime și criteriile de succes la începutul fiecărei etape, spunând, de exemplu, „după încărcarea fișierului de imagine, sistemul trebuie să afișeze estimarea prețului în termen de două secunde” sau „optimizarea poziționării foilor va reduce deșeurile cu cel puțin 5% comparativ cu metoda manuală.” Urmând raționamentul, am testat rapid aplicația într-un mediu local, am încercat mai multe cazuri de utilizare și am verificat integritatea rezultatelor. Dacă găsim probleme de performanță sau erori de calcul, le corectam imediat în același ciclu fără a schimba arhitectura generală sau structura codului.

Un mic jurnal pe calculator mi-a susținut procesul în timp ce documentam progresul fiecărui proiect: „De implementat,” „În testare,” și „Finalizat.” Această vizualizare m-a menținut concentrat și a prevenit acumularea de funcții parțial implementate sau netestate. Am învățat să îmi studiez propriul cod de mai multe ori, să creez comentarii clarificatoare și să înregistrez pașii semnificativi ai proiectului, chiar dacă nu aveam o echipă cu care să fac

recenzii de cod, doar niste cunostinte care ma sfatuiau ocazional. Am ținut, de asemenea, in acelasi jurnal, problemele întâmpinate și soluțiile decise, astfel încât să reflectez asupra modului în care am gestionat fiecare dintre ele într-o ordine logică.

Procesul iterativ mi-a permis să combin rapid comentariile mele cu ideile recomandate de coordonatorul lucrării de licență. Chiar dacă a implicat revizuirea implementărilor anterioare, am arătat progresul lucrării printr-o demonstrație scurtă după fiecare iterație. Acest exercițiu m-a ajutat să definesc ideile fiecărui modul și să modific cerințele funcționale în cazul în care constatam că soluția utilizată nu satisface complet nevoile industriei de ambalare.

Practic, modelul iterativ-incremental a însemnat că toate componentele importante, interfața de încărcare a imaginilor, algoritmul de calculare a dimensiunilor, logica de optimizare și generarea SVG au fost construite, testate și integrate treptat pentru a garanta că, în cele din urmă, am produs un prototip coerent și complet funcțional. Respectând termenele și criteriile academice, am reușit să mențin o viteză de dezvoltare constantă, să reduc riscurile de regresie și să produc o lucrare finală gata pentru susținerea tezei.

3.2. Etapele dezvoltării aplicației

Dezvoltarea aplicației a fost împărțită în patru etape majore:

Etapa 1: **Configurarea mediului de dezvoltare și proiectarea aplicației**

- Alegerea tehnologiilor de dezvoltare: PHP, HTML/CSS, JavaScript, MySQL;
- Definirea structurii logice a aplicației (frontend, backend, module pentru clienți și firmă);
- Proiectarea bazei de date (tabele pentru utilizatori, comenzi, fișiere încărcate);
- Pregătirea mediului de testare local și implementarea primei versiuni de interfață.

Etapa 2: Implementarea funcționalităților pentru client

- Modul de autentificare și înregistrare client;
- Funcționalitatea de încărcare a unei imagini cu obiectul pentru care se dorește ambalaj;
- Opțiunea de introducere manuală a dimensiunilor;
- Validarea datelor și trimiterea comenzii către sistemul backend;
- Afișarea mesajelor de confirmare sau eroare.

Etapa 3: Funcționalități pentru firmă și optimizarea materialului

- Implementarea unei interfețe de administrare pentru firmă;

- Calcularea automată a coalei necesare și generarea modelului de ambalaj cu risipă minimă;
- Salvarea comenzilor într-o bază de date și afișarea acestora într-o interfață tabelară;
- Pregătirea proiectului de tăiere în format compatibil cu utilaje CNC.

Etapa 4: Testare, validare, îmbunătățiri și pregătirea pentru livrare

- Testare manuală și automată a fiecărui modul;
- Corectarea erorilor și optimizarea codului;
- Îmbunătățirea interfeței grafice: butoane clare, mesaje explicite, navigare intuitivă;
- Validare finală împreună cu utilizatori reali (simulări de comenzi complete);
- Redactarea documentației tehnice și a ghidului de utilizare.

3.3. Concluzii privind modelul de dezvoltare

Modelul incremental, completat cu practici Agile, a oferit o soluție echilibrată între controlul dezvoltării și flexibilitate. Alegerea acestui model a permis construirea treptată a unei aplicații stabile și intuitive, cu testare constantă, validare progresivă și integrare de feedback. Modelul a favorizat comunicarea continuă cu utilizatorii și adaptarea la observațiile primite, iar rezultatul final reflectă o implementare coerentă și eficientă.

Diagrama de mai jos ilustrează cele patru incrementuri definite în cadrul proiectului: primul include autentificarea și înregistrarea, al doilea gestionează uploadul imaginii și estimarea dimensiunilor, al treilea permite configurarea comenzii, iar al patrulea oferă firmelor acces la interfață și funcționalități de optimizare a materialului.

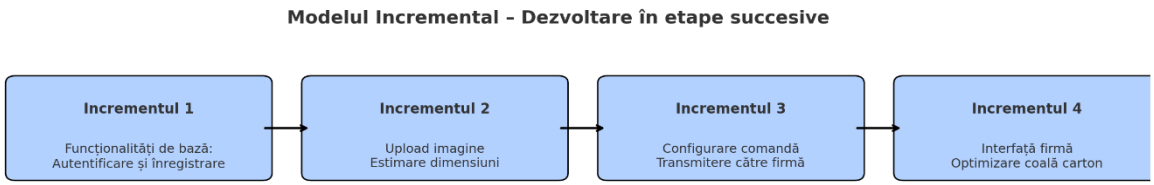


Figura 3: Modelul Incremental

Proiectarea sistemului informatic

În acest capitol, voi trece în revista cum am transformat criteriile și procesele menționate anterior într-o arhitectură software coerentă, modulară și ușor de întreținut. De la sistemul de autentificare și gestionarea sesiunilor până la încărcarea imaginilor și calculul automat al dimensiunilor, optimizarea colilor ("nesting") și producția de fișiere SVG pentru utilaj CNC, voi prezenta pe rând toate componentele cheie ale aplicației. Apoi voi detalia alegerea acestor modelele de design și alegerile tehnologice, voi clarifica structura de bază a codului și voi arăta cum interacționează fiecare modul atât în interiorul său, cât și în afara sa.

Am lucrat in PHP vanilla pe partea serverului, abilitățile pe care le-am dobândit la facultate m-au ajutat să construiesc rapid cod curat fără a adăuga alte dependențe care ar putea complica instalarea sau întreținerea ulterioară. Am aranjat fișierele, scripturi de logică de afaceri, API-uri interne AJAX, gestionarea sesiunilor, fișiere de configurare, în directoare care reflectă diviziunea responsabilităților. Această arhitectură îmi garantează controlul complet asupra fiecărui element al aplicației și îmi permite să adaug rapid noi funcționalități în cazul în care apar nevoi suplimentare.

Din punct de vedere al interfeței, am folosit HTML, CSS și puțin JavaScript . Fiecare element vizual din design-ul simplu și eficient răspunde imediat la mișcările utilizatorului. JavaScript personalizat mi-a permis să efectuez validări la cerere, să afișez estimări de preț în timp real după procesarea imaginilor și să gestionez dinamica interfeței cu usurinta.

Pentru baza de date am folosit o metodă hibridă combinând MySQL cu fișiere CSV locale pentru a stoca datele. Gestionând în mod fiabil creddentialele utilizatorilor, istoricul comenzilor și oferind integritate referențială, tranzacționalitate și capacitatea de căutări sofisticate, baza de date relațională MySQL oferă de asemenea o centralizare eficienta si optiune de scalabilitate pe viitor. Deoarece beneficiarii companiei utilizează deja macrocomenzi, foi de calcul și alte instrumente care procesează fișiere CSV, formatele CSV sunt metoda principal folosita de beneficiar pentru manipularea comenzilor. Prin alegerea aceasta am reusit sa le ofer comfort in utilizarea datelor, fara a fi nevoie sa exporte din MySQL sau sa fie o curba de invatare a sistemului, informatiile de care au nevoie sunt stocate direct cum sunt obisnuiti.

Am inclus, de asemenea, servicii terțe și biblioteci specializate pentru estimarea folosind inteligența artificială. În timp ce bibliotecile specializate în generarea SVG garantează fișiere gata de transferat către sistemul CNC, apelurile către API-ul OpenAI permit identificarea automată a formelor și calcularea exactă a dimensiunilor.

În concluzie, prin această abordare am reușit să transpun criteriile și fluxurile de lucru identificate într-o arhitectura solidă și extensibilă. Alegerea PHP mi-a permis un control asupra codului server, organizat pe directoare clare, iar combinația consacrată HTML-CSS-JavaScript s-a dovedit suficientă pentru o interfață reactivă și intuitivă. Strategia hibridă de stocare a datelor îmbină faibilitatea și posibilitatea de extindere cu familiaritatea personalului, iar integrarea inteligenței artificiale aduce un bonus și un punct bun de dezvoltare pe viitor, pe masură ce apar modele mai eficiente de recunoaștere a imaginilor. Algoritmii de optimizare de coala aduc un plus valoare real benficiarului, reducand risipa și crescând marja de profit.
În ansamblu, această etapă asigura baza tehnică necesară pentru pașii următori de detaliere a proceselor.

4. Proiectarea logică

Fără a aborda încă specificul implementării fizice, în etapa de proiectare logică ne concentrăm pe descrierea structurii datelor în termeni de entități și atribute. „Cea mai importantă motivație pentru lucrările de cercetare care au dus la modelul relațional a fost obiectivul de a oferi o delimitare clară și precisă între aspectele logice și cele fizice ale gestionarii bazelor de date” (Codd, 1970). Am definit două tabele majore MySQL pentru acest prototip, fiecare reflectând o necesitate evidentă a aplicației. Strict limitat la ceea ce este necesar pentru autentificare și roluri, tabela utilizatorilor constă dintr-o identificare internă, adresa de email asigurată a fi unică și obligatorie, parola păstrată criptată și un câmp pentru rol (admin sau client). Această arhitectură simplă garantează că fiecare utilizator are un cont sigur disponibil în orice moment pentru includerea unor niveluri suplimentare de drepturi sau a unor noi atribute de profil. Pentru gestionarea datelor relaționale și implementarea tabelelor users și comenzi, aplicația folosește sistemul de gestiune MySQL, documentat oficial de Oracle Corporation (2024).

Tabelul comenzilor aranjează informațiile despre nevoile de ambalare: data și ora plasării, numele fișierului încărcat (dacă există), dimensiunile în milimetri, grosimea cartonului și numarul de bucati. Procesul de comanda este restrictionat de ologarea cu un cont valid.

Fiecare rând din tabelul de comenzi este exportat simultan într-un fișier comenzi.csv, unde coloanele se potrivesc exact cu structura relațională, permițând astfel organizației să reacționeze rapid la fluxul său operațional. Cu integritatea referențială și indicii de performanță, această strategie hibridă păstrează baza de date MySQL pregătită pentru un transfer total către

o soluție pur relațională, oferind în același timp compatibilitate rapidă cu instrumentele actuale (foi de calcul, soft-uri terte de analiza).

Astfel, prin menținerea unei scheme logice clare și normale și printr-o strategie de stocare hibridă, proiectul combină eleganța modelului relațional cu flexibilitatea necesară pentru un prototip demonstrativ, deschizând astfel calea pentru extinderi viitoare fără a compromite integritatea și coerența datelor actuale.

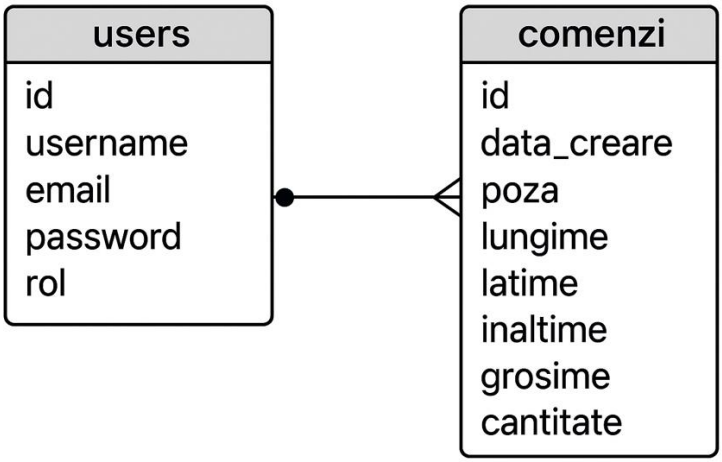


Figura 4: Tabele SQL

4.1. Arhitectura sistemului informatic

Definiția oficială a standardului IEEE 42010 spune: „Arhitectura software este structura sau structurile sistemului, care cuprind elemente software, proprietățile vizibile extern ale acestor elemente și relațiile dintre ele”. Din acest punct de vedere, arhitectura arată nu numai componentele sale, ci și modul în care acestea interacționează și influențează unul pe celălalt. Acest lucru face arhitectura ca o hartă de navigare prin complexitatea oricărui sistem software.

Folosind aceste concepte, am împărțit aplicația de configurare a ambalajului în trei niveluri distincte, fiecare având funcții distincte. Paginile web, care sunt construite cu scripturi JavaScript și sunt proiectate în HTML și CSS, garantează validări, previzualizări și afișări dinamice ale estimărilor de prețuri fără blocare, acționând astfel ca un intermediar între utilizator și mecanismele interne care nu sunt vizibile. Stratul central al logicii de afaceri, care este implementat în mod complet în varianta „vanilla”, coordonează fluxurile principale, cum ar fi autentificarea, gestionarea sesiunilor, calcularea algoritmului de optimizare, trimiterea și

primirea datelor de la serviciul de inteligență artificială pentru măsurarea obiectelor și crearea de fișiere SVG pentru mașinile CNC. Fiecare element este organizat într-un mod distinct și ușor extensibil. În primul rând, stratul de date combină integritatea oferită de o bază de date relațională MySQL cu accesibilitatea și familiaritatea fișierelor CSV utilizate de operatorii de producție. Această dublă persistență permite manipularea rapidă a foilor de calcul și nevoile de centralizare și analiză a datelor.

Acest design cu mai multe fațete face ca fiecare parte să fie un bloc independent de construcție. De exemplu, este posibil să reinterpretați interfața într-un framework diferit, logica să se mute către microservicii și persistența datelor să se transforme treptat într-o soluție de tip afaceri fără a afecta coeziunea întregului sistem. Deciziile arhitecturale garantează că proiectul rămâne adaptabil, testabil și pregătit pentru schimbare în ansamblu, așa cum sugerează profesioniștii în arhitectura software. Un design bun nu numai că rezolvă problemele actuale, dar creează și calea pentru o dezvoltare rapida.

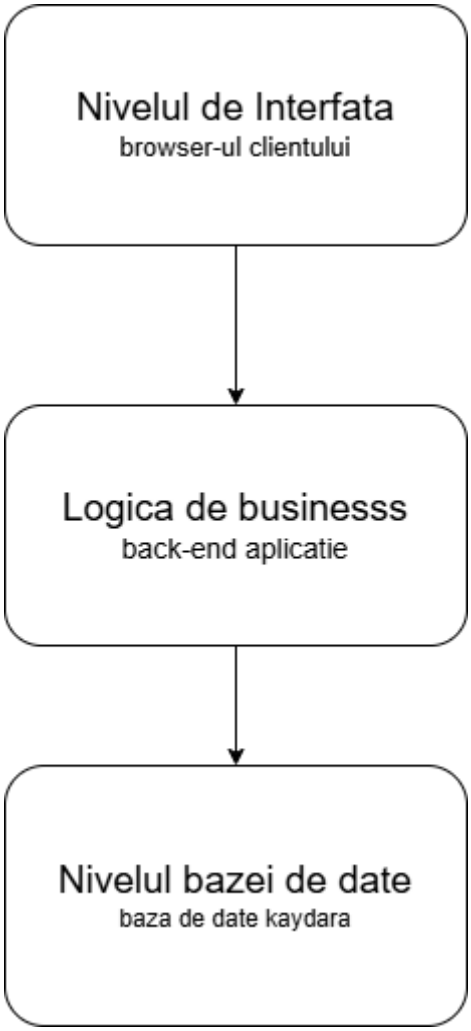


Figura 5: Structură multi-layer

5. Proiectarea Tehnică

În acest capitol voi incapsula structura generală a proiectului, forma datelor și logica urmarită, dar și tehnologiile și motivatia alegerii lor.

5.1. Structura generală

Bazat pe două domenii funcționale separate -'public/', destinat interfeței utilizatorului final, și 'admin/', destinat gestionării și întreținerii datelor din back-office, proiectul este împărțit într-o arhitectură de bază, dar eficientă. Gestionarea clară a mai multor procese este posibilă datorită acestei separări: operatorii interni autentificați se ocupă de comenzile salvate, au acces la date precum varianta optima de productie si statistici de risipa si eficienta și creează fișiere SVG pentru producție; utilizatorii neautorizați accesează doar rezumatul paginilor publice și pot face comenzi, daca sunt logati cu un cont de utilizator. Structura aplicației a fost realizată conform specificațiilor oficiale ale World Wide Web Consortium pentru HTML5, CSS3 și SVG (World Wide Web Consortium, 2017, 2018).

Toate fișierele necesare pentru experiența finală a clientului se găsesc în directorul public: paginile statice ("Avantaje," "Despre noi") care descriu serviciul, cadrul principal al configuratorului de dimensiuni (client.php), scripturile de validare și comunicare asincronă ("estimate.php") pentru estimarea automată a dimensiunii prin API-ul OpenAI, precum și fișierul salveaza_comanda.php care recuperează datele din formular și le trimite pentru persistență. În timp ce logout.php marchează sfârșitul sesiunii, paginile login.php și register.php se ocupă de configurarea și validarea contului, gestionând astfel autenticitatea. Elementele comune ale interfeței, header și footer, sunt scoase în fișiere independente (Header.php, footer.php), incluse în toate paginile in cae sunt solicitate, pentru consistență vizuală și navigare.

Directorul 'admin' contine focusul pe partea de business, functiile de optimizare, interfata de afisare a comenzilor si beneficiile programului. Tabelul de comenzi din zona business afiseaza date precum data si ora comenzii, dimensiunile obiectului, cantitatea, coala optima de folosit in procesul de productie pentru comanda respectiva, optiunea de a vizualiza si descarca proiectul gata de trimis catre utilaj si chiar vizualizarea pozei incarcata de client daca acesta a folosit functionalitatea de estimare cu inteligenta artificiala in cadrul configuratorului.Comenzile sunt pastrate in sistem hibrid, ca tinta pricipala ele sunt transmise

intr-un fisier csv local pentru usurinta de lucrare cu datele din partea personalului, a fost o cerere de-a lor, dar sunt trimise si intr-o baza de date MySQL intr- un tabel Comenzi. Imaginile încărcate de clienți sunt păstrate în folderul uploads/, iar scripturile precum optimize.php furnizeaza cea mai buna varianta de productie din punct de vedere al materiei prime si al colilor și arată detalii precum numarul necesar si bucati/coala. Fisierul generate_svg.php convertește ulterior planul geometric într-un fișier SVG potrivit pentru importul în mașinile CNC. Asigurând un singur punct de configurare pentru toate scripturile de back-office, fișierul db.php inițiază conexiunea PDO la MySQL și specifică restricțiile de acces la baza de date.

Bazat pe include și require, acest cadru procedural menține lucrurile simple: fiecare fișier PHP își asumă o singură sarcină, iar includerea logicii și interfețelor se face manual, dacă este necesar. De la front-end la persistența datelor și până la optimizarea proceselor de fabricație, organizarea în foldere și fișiere reflectă fluxul natural al programului, oferind astfel claritate.

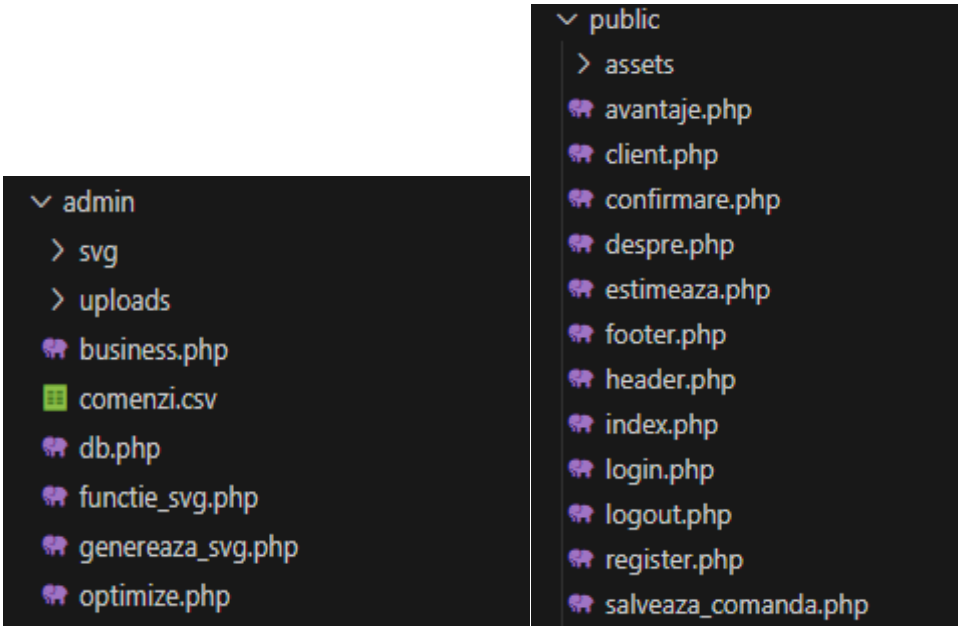


Figura 6: Structura proiectului în mediul de lucru

5.2. Structura fizică a datelor

În momentul de față, persistența informațiilor în aplicație se bazează pe o combinație explicată și justificată de două mecanisme: stocarea relațională – reprezentată de tabelele users și comenzi din MySQL – și jurnalizarea plată într-un fișier CSV aflat în directorul admin. Alegerea nu este întâmplătoare: baza de date oferă tranzacționalitate, integritate și interogări

flexibile, în timp ce fișierul text răspunde nevoii de manipulare imediată a datelor în mediul de producție, unde operatorii preferă instrumente de calcul tabelar.

Tabela users reflectă doar ceea ce este strict necesar pentru autentificare și diferențiere de roluri. Am optat pentru un câmp username unic și limitat la 50 de caractere, suficient pentru nume descriptive, dar nu exagerat încât să ocupe spațiu inutil în index. Parolele nu sunt reținute în clar, ci ca hash-uri bcrypt stocate în passhash; lungimea de 255 de caractere acoperă marginile oricărui algoritm de hash modern rulat prin password_hash(). Coloana role, definită ca ENUM('client','admin'), le permite scripturilor de securitate să verifice drepturile printr-o simplă comparație textuală, reducând overhead-ul unui join cu o tabelă de roluri. Chiar dacă e o soluție minimală, ENUM-ul rămâne ușor de extins: adăugarea unui nou rang se face printr-un singur ALTER TABLE, fără migrații complexe. În mod deliberat, nu am impus un e-mail obligatoriu: în etapa de prototip s-a considerat că login-ul pe bază de nume de utilizator este suficient, iar colectarea obligatorie a adresei ar fi încetinit procesul de înregistrare.

Tabela comenzi adună parametrii tehnici ai fiecărei cutii. Dimensiunile – lungime, lățime, înălțime – sunt salvate ca INT UNSIGNED, exprimate direct în milimetri; am renunțat la DECIMAL tocmai pentru a evita conversiile suplimentare și pentru că unitatea fixă elimină ambiguitățile. Grosimea cartonului și cantitatea folosesc același tip, iar coloana poza păstrează doar numele fișierului, nu și calea absolută Timestamp-ul comenzii, data_creare, se completează automat cu CURRENT_TIMESTAMP(), oferind un istoric cronologic și permițând interfației de administrator să sorteze rapid după dată, cu o simpla functionalitate integrata. Ca index secundar, a fost creat un singur indice pe data_creare; pentru traficul curent, acesta este suficient, dar schema rămâne deschisă pentru eventuale indexări pe dimensionale sau combinații (de pildă, (grosime_mm, cantitate)) dacă volumul crește.

În paralel cu inserția SQL, aceeași comandă este înscrisă, prin funcția fputcsv(), în fișierul admin/comenzi.csv. Faptul că apelăm o funcție standard PHP ne asigură citarea automată a câmpurilor care conțin virgule sau spații și, implicit, compatibilitate cu orice program capabil să deschidă CSV-uri UTF-8. Coloanele sunt dispuse în exact aceeași ordine ca în tabelă, precedate de data comenzii, pentru a permite, la nevoie, re-importul direct prin LOAD DATA. Fișierul rămâne deocamdată deschis la infinit; nu există un cron de rotație sau de curățare, iar decizia este conștientă: prototipul trebuie să demonstreze funcționarea, nu să rezolve încă probleme de arhivare. Dacă dimensiunea fișierului va deveni incomodă, rotația poate fi adăugată printr-un script minimal care să redenumească logul la sfârșitul fiecărei luni. De asemenea, orice politici de backup rămân la latitudinea echipei de operare, dar soluția

actuală – SQL dump manual și copiere periodică a CSV-ului - este considerată suficientă pentru mediul de test.

Prin urmare, designul fizic rămâne intenționat sobru: două tabele MySQL in baza de date Kaydara, un fișier CSV și un director de upload pentru imagini. Această simplitate garantează pornire rapidă, depanare facilă și, mai ales, un drum clar pentru extindere. Dacă aplicația va ajunge într-un context de producție industrială, putem introduce relația directă între comenzi și users, politici automate de arhivare sau chiar partitionare pe ani; până atunci, schema actuală servește perfect scopului de prototip demonstrativ, îmbinând securitatea datelor cu accesibilitatea nemijlocită cerută de atelier.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
1	**id**	int(11)			No	*None*		AUTO_INCREMENT
2	**username**	varchar(50)	utf8mb4_general_ci		No	*None*		
3	**passhash**	varchar(255)	utf8mb4_general_ci		No	*None*		
4	**email**	varchar(120)	utf8mb4_general_ci		Yes	*NULL*		
5	**role**	enum('client', 'admin')	utf8mb4_general_ci		Yes	client		

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
1	**id**	int(11)			No	*None*		AUTO_INCREMENT
2	**data_creare**	datetime			Yes	current_timestamp()		
3	**poza**	varchar(255)	utf8mb4_general_ci		Yes	*NULL*		
4	**lungime_mm**	int(11)			No	*None*		
5	**latime_mm**	int(11)			No	*None*		
6	**inaltime_mm**	int(11)			No	*None*		
7	**grosime_mm**	int(11)			No	*None*		
8	**cantitate**	int(11)			No	*None*		

Figură 7: Tabele SQL

5.3. Fluxurile de date

Fluxul informațional reprezintă traseul pe care îl parcurg datele din momentul în care utilizatorul inițiază o acțiune și până la persistența lor definitivă în sistem. În configuratorul de ambalaje, acest traseu este împărțit în două secvențe complementare: o secvență opțională de estimare automată a dimensiunilor și o secvență obligatorie de trimitere și salvare a comenzii.

Separarea celor două etape permite, pe de o parte, degrevarea serviciului extern de solicitări inutile și, pe de altă parte, menținerea clară a responsabilităților fiecărui modul.

5.3.1. Estimarea automată

Utilizatorul poate solicita o pre-estimare a dimensiunilor obiectului pe care dorește să-l ambaleze. Cererea pornește din browser: fotografia este validată la nivel client (format, mărime) și transmisă, printr-un apel AJAX, spre scriptul estimeaza.php. Aici, fișierul este verificat din nou server-side pentru a preveni încărcarea de conținut malițios, apoi imaginea este codificată și expediată către API-ul OpenAI. Serviciul extern analizează cadrul vizual și întoarce un răspuns JSON cu lungimea, lățimea și înălțimea obiectului în pixeli. Scriptul convertește aceste valori în milimetri, pe baza unui factor de scalare configurat, și le retrimite browser-ului. Interfața completează automat câmpurile formularului; utilizatorul le poate accepta ca atare sau le poate ajusta manual. Dacă serviciul AI eșuează ori răspunsul nu trece filtrele de plauzibilitate, aplicația revine elegant la datele introduse de utilizator, marcând incidentul în jurnalul de erori fără a întrerupe sesiunile active.

5.3.2. Trimiterea și salvarea comenzii

Indiferent dacă a folosit sau nu estimarea automată, utilizatorul finalizează formularul și apasă „Trimite comanda”. Datele – dimensiuni, grosime, cantitate și, dacă există, fotografia – sunt trimise din nou prin AJAX, de această dată către scriptul salveaza_comanda.php. Fișierul bootstrap index.php inițializează sesiunea, încarcă configurația globală și direcționează cererea spre scriptul dedicat. În salveaza_comanda.php, datele sunt validate riguros: numerele trebuie să fie pozitive, cantitatea să depășească zero, iar tipul imaginii să se încadreze în lista permisă. Fotografia este mutată în directorul admin/uploads/, sub un nume unic, pentru a preveni coliziunea cu fișiere existente.

Odată validate datele, aplicația apelează optimize.php, modulul care calculează modul optim de așezare a decupajelor pe coală. Algoritmul returnează numărul de coli necesare, suprafața irosită și procentul de eficiență. Rezultatele sunt apoi persistate simultan în două depozite: un INSERT tranzacțional completează tabela comenzi din MySQL, iar funcția fputcsv() adaugă un rând în fișierul admin/comenzi.csv. Această dublă scriere garantează atât siguranța datelor (prin baza relațională), cât și accesibilitatea rapidă pentru operatorii care lucrează în foi de calcul.

Ultimul pas al procesării server-side îl constituie invocarea modulului genereaza_svg.php, care transformă geometria cutiei în comenzi SVG conforme cu utilajele CNC ale firmei. Fișierul rezultat este salvat temporar pe server, iar adresa sa este inclusă în răspunsul JSON trimis browser-ului. Interfața actualizează imediat rezumatul comenzii: cost, eficiență, link de descărcare a SVG-ului și un mesaj de confirmare.

5.4. Tehnologii specifice

Prototipul a fost dezvoltat în **Visual Studio Code**, editor de cod ușor și gratuit care, „dincolo de funcțiile de editare și depanare standard, include suport IntelliSense, control Git integrat și o piață vastă de extensii", facilitând tranziția rapidă de la idee la cod executabil Visual Studio Code. Prin comparație cu IDE-urile tradiționale, footprint-ul redus și pornirea instantanee au contat decisiv într-un proiect de licență derulat pe hardware modest.

Pentru logica de server a fost ales PHP 8. Noua versiune aduce îmbunătățiri notabile – operatorul nul-safe, expresia match, JIT – care „simplifică sintaxa și reduc latența execuției" conform notei oficiale de lansare php.net. Limbajul, nativ orientat spre web, se îmbină natural cu markup-ul HTML și este disponibil pe orice găzduire LAMP, menținând costurile operaționale la un nivel accesibil.

Interfața utilizator este construită pe HTML5**,** CSS3 **și** JavaScript. Standardele W3C subliniază că aceste tehnologii formează „scheletul, prezentarea și interactivitatea paginilor moderne" W3C. În locul unui framework SPA s-a optat pentru JavaScript vanila, suficient pentru validările client-side și solicitările Fetch, eliminând dependențele suplimentare și menținând pachetul livrat sub câteva zeci de kilobyți.

Persistența relațională se sprijină pe MySQL 8, a cărui componentă InnoDB „respectă strict modelul ACID, protejând datele împotriva corupției și asigurând recuperarea după căderi" MySQL Developer Zone. Tabela users stochează conturile în format hash, iar tabela comenzi păstrează istoricul solicitărilor; indexarea pe dată permite rapoarte rapide. Pentru uzul imediat al operatorilor, fiecare comandă este serializată în paralel într-un fișier CSV, format text plat care poate fi deschis instant în Excel sau LibreOffice Calc – soluție „low-tech", dar perfect adaptată fluxului din hală.

Conversia logicii geometrice în fișier de producție se realizează direct în PHP, prin clasa DOMDocument, „reprezentarea unui întreg document XML/HTML” din bibliotecile standard php.net. Construirea programatică a nodurilor <svg> permite adaptarea conturului la orice dimensiune fără șabloane statice și înlătură dependențele de biblioteci externe de grafică.

Componenta de inteligență artificială exploatează OpenAI Vision API – un serviciu REST care „permite modelelor să analizeze imagini și să extragă informații cuantitative” OpenAI Community. Integrarea se rezumă la trimiterea unui payload multipart și la interpretarea unui răspuns JSON.

Prin îmbinarea unui IDE agil, a stivei LAMP, a front-end-ului standard și a unui serviciu AI de ultimă generație, prototipul oferă un echilibru între timp scurt de dezvoltare, cost redus și fundație solidă pentru extindere spre cerințe industriale mai complexe.

6. Implementarea aplicației

În următorul capitol voi pune în lumină procesele importante ale aplicației prin intermediul cărora se realizează funcționalitățile principale, exemplificând atât algoritmii și metodele folosite, dar și dificultățile întampinate.

6.1. Configuratorul de cutie

Combinând un design aerisit cu sisteme inteligente de validare și estimare automată, configuratorul de cutii reflectă ecranul principal al experienței clientului. Existența unei sesiuni valide este verificată din momentul accesării paginii: dacă utilizatorul nu este autentificat, i se afișează un avertisment clar cerându-i să se conecteze înainte de a putea continua. Prin intermediul acestei securități la nivel de server, securitatea este întărită și accesul ilegal este oprit, păstrând astfel integritatea procesului de comandă.

Vizitatorul găsește un formular structurat în trei faze logice odată ce intră în configurator. În prima parte, „Încarcă o imagine,” utilizatorul poate furniza o fotografie a obiectului destinat ambalării. Includerea unei referințe cu dimensiuni cunoscute, exemplu o monedă, o riglă, lângă obiect va ajuta serviciul de inteligență artificială să facă o estimare precisă. Browserul arată un preview al imaginii pe pagina curentă imediat după ce fișierul a fost ales, oferind astfel feedback rapid și verificând încărcarea corectă. Un mesaj mic sub buton

informează utilizatorul simultan despre semnificația obiectului de referință pentru acuratețea estimării.

A doua parte, „Estimare Dimensiuni," include un buton destinat să trimită imaginea către un script specific. Programul comunică cu API-ul de viziune OpenAI în fundal: imaginea este încărcată în cloud, analizată și convertită într-un răspuns structurat cu valorile lungimii, lățimii și înălțimii obiectului exprimate în milimetri. Interfața completează automat câmpurile de dimensiune la primirea răspunsului, permițând utilizatorului să le mențină sau să le schimbe personal. Dacă serviciul extern este lent, eșuează sau produce rezultate improbabile, aplicația revine subtil la datele introduse manual fără a opri fluxul și fără a produce erori evidente.

În ultima etapă, „Detalii Comandă", clientul introduce numărul dorit de cutii și grosimea cartonului. Pe baza suprafeței desfășurate a cutiei și a tarifelor ajustate în funcție de grosime, un calculator intern actualizează costul estimat în timp real cu orice modificare. Rezultatul apare imediat sub formular într-o secțiune separată, atât prețul total al comenzii, cât și prețul unitar pe cutie, permițând astfel utilizatorului să evalueze efectul fiecărei modificări fără a reîncărca pagina.

Clientul face clic pe butonul „Trimite Comanda" după validarea completă a fiecărui câmp. Formularul este trimis la server folosind o tehnică tipică multipart POST și pagina nu se reîncarcă. Prin urmare, experiența rămâne impecabilă și ușoară, cu feedback vizual continuu și un accent pe minimizarea încărcărilor suplimentare ale paginii.

Fiecare componentă a configuratorului, de la plasarea butoanelor până la culorile selectate pentru mesaje și stările de eroar, a fost concepută pentru a ghida utilizatorul cu precizie. Oferă o interfață prietenoasă, puternică și rapidă, în combinație cu validări pe partea clientului și pe partea serverului, precum și soluții de rezervă pentru evenimente neprevăzute, reducând astfel timpul de plasare a comenzii la câteva zeci de secunde. Apoi vom discuta despre componenta de persistență și optimizare a materialelor, care transformă datele introduse în comenzi potrivite pentru fabricație.

Pentru a estima costul materialului aferent unei comenzi, scriptul JavaScript calculează mai întâi suprafața desfășurată a unei cutii: lungimea externă plus lățimea externă (fiecare extinsă cu de două ori grosimea cartonului) formează perimetrul, iar înălțimea este mărită cu două clapete de pliere, rezultând o zonă plană care va fi decupată din coală. Această suprafață, exprimată inițial în milimetri pătrați, este convertită în metri pătrați prin împărțire la un milion. Ulterior, funcția de tarifare determină prețul pe metru pătrat în funcție de grosimea cartonului, aplicând praguri scalate (de exemplu, carton sub 2 mm la 3,5 lei/m², sub 6 mm la 5,5 lei/m², etc.). În final, înmulțirea suprafeței totale (m²) cu tariful corespunzător produce costul estimativ

al materialului, care se împarte, după preferinţe, fie pe unitatea de comandă, fie pe fiecare cutie individual.

6.1.1. Determinarea suprafeței desfășurate:

Pornind de la dimensiunile interne introduse de utilizator: lungime L, lățime W și înălțime H (toate în milimetri) , se definesc mai întâi dimensiunile exterioare:

$$L_{\text{ext}} = L + 2\,g, \quad W_{\text{ext}} = W + 2\,g, \quad H_{\text{ext}} = H + 2\,g$$

Figura 8: Calcul 1

Suprafața desfășurată rezultă din produsul lungimii externe și înălțimii de tăiere, care la rândul său ține cont de clapetele laterale:

$$P = 2\big(L_{\text{ext}} + W_{\text{ext}}\big), \quad H_{\text{corp}} = H_{\text{ext}} + 2\,\max\big(L_{\text{ext}}, W_{\text{ext}}\big),$$

$$A_{\text{cutie}}^{\text{mm}^2} = P \times H_{\text{corp}}, \quad A_{\text{cutie}}^{\text{m}^2} = \frac{A_{\text{cutie}}^{\text{mm}^2}}{1\,000\,000}.$$

Figura 9: Calcul 2

6.1.2. Aplicarea tarifului in funcție de grosime:

Pentru a reflecta diferențele de cost între carton subțire și carton rigid, tariful pe metru pătrat p este ales din funcție de grosime:

$$p = \begin{cases} 3{,}5\ \text{lei/m}^2, & g \le 2\ \text{mm}, \\ 4{,}5\ \text{lei/m}^2, & 2 < g \le 4\ \text{mm}, \\ 5{,}5\ \text{lei/m}^2, & 4 < g \le 6\ \text{mm}, \\ 6{,}5\ \text{lei/m}^2, & g > 6\ \text{mm}. \end{cases}$$

Figura 10: Calcul 3

6.1.3. Calculul costului total si unitar:

Dacă utilizatorul comandă q cutii, costul total este:

$$\text{Cost total} = A_{\text{cutie}}^{\text{m}^2} \times p \times q,$$

Figura 11: Calcul 4

Iar costul per cutie se obtine simplu ca:

$$\text{Cost per cutie} = A_{\text{cutie}}^{\text{m}^2} \times p.$$

Figura 12: Calcul 5

În cele din urmă, mecanismul de estimare a costului integrat în configurator combină formule matematice simple cu praguri de tarifare pragmatice pentru a oferi o vizibilitate instantanee și transparentă a efectelor alegerilor de grosime, dimensiuni și cantitate. Clientul poate modifica parametrii comenzii pentru a optimiza bugetul prin calcularea suprafeței desfăşurate și aplicarea imediată a prețului pe metru pătrat. Interfața funcționează rapid, punând responsabilitatea de decizie în mâinile utilizatorului. Prin urmare, asigură atât acuratețea estimărilor, cât și o experiență de utilizare fluidă, ceea ce reduce nevoia de consultări suplimentare și crește încrederea în procesul de achiziție încă din faza de prototip.

6.2. Integrarea API-ului extern OpenAI

"Estimează Dimensiunile", componenta care adaugă inteligență artificială configuratorului, a fost concepută să fie cât mai fluidă și mai puțin intruzivă posibil pentru utilizator. Serverul verifică rapid prezența și integritatea fișierului încărcat atunci când apasă butonul dedicat, apoi codifică imaginea în format Base64 pentru a o încorpora direct într-o cerere JSON. Această cerere este transmisă endpoint-ului de viziune OpenAI și este însoțită de un prompt bine calibrat care îi indică modelului să returneze doar cele trei dimensiuni în milimetri, fără alte comentarii. Apoi, o functie de expresie este folosita pentru a filtra răspunsul text finit și concis. Aceste trei valori numerice sunt extrase și afișate automat de interfață în câmpurile de lungime, lățime și înălțime. În cazul în care AI-ul nu oferă un răspuns adecvat dintr-o varietate de motive, aplicația folosește o soluție elegantă, lăsând câmpurile editabile manual și informând clientul printr-o notificare succintă.

Integrarea cu OpenAI Vision API s-a dovedit extrem de economică din punct de vedere al cheltuielilor. Un apel de estimare a dimensiunilor costă sub 0,01 USD, ceea ce înseamnă că impactul financiar este mic în situațiile în care serviciul e folosit de câteva zeci sau sute de ori pe zi. În plus, latența de procesare medie este de 600 până la 800 ms, ceea ce este o viteză suficientă pentru a evita întârzierile inacceptabile. Modulul oferă un plus de productivitate și confort, fără a afecta bugetul operațional al companiei, prin această combinație între engineering prompt, validări riguroase și costuri minime per interogare.

6.3. Algoritm de optimizare a consumului de materie primă

În industria ambalajelor din carton ondulat, eficiența utilizării materialelor este esențială atât din punct de vedere financiar, cât și din punct de vedere ecologic. Reducerea materialului inutil și îmbunătățirea ergonomiei ambalajului sunt obiective esențiale în orice proces de proiectare (Robertson, 2012). Din cauza consumului excesiv de resurse și a costurilor de producție crescute pentru fiecare coala de carton neutilizată pentru tăiere, componenta de "nesting" este un pas vital in cadrul oricarei firme de productie de amabalaje. Standardul mondial in modele de cutii este FEFCO(“Fédération Européenne des Fabricants de Carton Ondulé”), care a stabilit diferite coduri de identificare pentru diferite cutii. Aplicația noastră folosește modelul FEFCO 201, cunoscuta informal ca si "cutie de depozitare", fiind modelul dominant in aproape orice industrie, care oferă o cutie cu patru panouri laterale similare și capace de lipire. Este de departe cea mai folosita si ceruta cutie in industrie, motiv pentru care a fost alesa pentru acest prototip de aplicatie. Optimizarea poziționării decupajelor pe coala de carton se bazează pe principii similare cu cele discutate în algoritmii de nesting bidimensionali (Li, 2015).

Fiecare configurație a cutiei FEFCO 201 include nu numai suprafețele principale (față, spate și laterale), ci și flape de lipire și închidere (de sus și de jos). Cele mai bune practici din industrie, care includ praguri pentru dimensiunea clapetei de adeziv, care sunt determinate de lungimea maximă de tăiere, iau în considerare necesitatea unor margini suficiente pentru aplicarea adezivului și pentru cea mai bună rezistență mecanică.

Folosind dimensiunile interne, grosimea cartonului și modul în care acesta poate fi aranjat pe coli de carton de dimensiuni convenționale, vom detalia ulterior calculele matematice necesare pentru a determina suprafața desfășurată a conturului FEFCO 201. Vom analiza formula pentru calculul clapetelor, metodele de orientare și rotire pe foaie, precum și

evaluarea eficienței și deșeurilor, oferind o metodă practică care produce o soluție eficienta cu deșeuri totale minime.

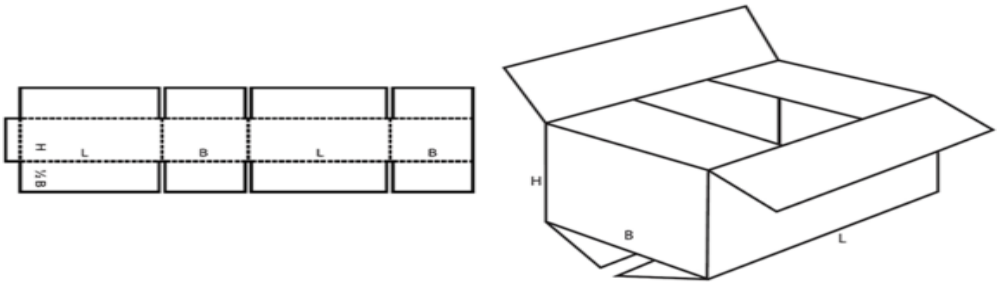


Figura 13: Standard Fefco 201

Optimizarea materialului (sau „nesting-ul”) reprezintă inima modulului de calcul al răspunsului pentru comandă: pornind de la dimensiunile interne ale cutiei şi grosimea cartonului, scopul este de a recomanda coală standard care produce cea mai mică pierdere de material. Funcţia de optimizare din codul PHP parcurs pas cu pas transformă datele de intrare într-o analiză completă, care cuprinde atât dimensiunile exacte ale desenului de tăiere, cât şi evaluarea detaliată a numărului de coli, a suprafeţei efectiv utilizate şi a deşeurilor rezultate.

În prima fază, dimensiunile interne ale cutiei, lungime, lăţime şi înălţime, sunt extinse cu grosimea materialului. Această ajustare reflectă comportamentul fizic al cartonului ondulat la îndoire, unde un plus de material asigură margini solide după pliere şi permite îmbinări curate. Astfel se definesc trei valori noi, care includ grosimea pe ambele feţe ale fiecărei laturi.

Următorul pas se concentrează pe elementele specifice modelului FEFCO 201: flapă de lipire şi clapete de închidere. Flapa, necesară pentru suprapunerea marginilor şi aplicarea adezivului, este dimensionată după lungimea maximă a desenului în plan, cu praguri practice (20, 30 sau 45 mm). Clapetele superioare şi inferioare, care asigură închiderea în siguranţă a

cutiei, se calculează ca jumătate din latura mai mică plus grosimea cartonului, iar apoi extinderea cu câţiva milimetri suplimentari asigură o suprapunere optimă şi o rezistenţă mecanică adecvată.

Odată conturul complet definit, funcţia parcurge lista preconfigurată de dimensiuni standard de coala de carton. Pentru fiecare dimensiune, se încearcă plasarea desenului atât în orientarea sa normală, cât şi rotit la 90°, pentru a determina câte contururi pot încăpea pe coală fără suprapunere. Această analiză exploatează diviziunile întregi ale dimensiunilor foii, folosind calcule de tip „floor” pe raportul dintre laturile coalei şi dimensiunile conturului. Rezultatul maxim dintre cele două variante defineşte numărul de contururi per coală.

În etapa finală, se calculează câte coli sunt necesare pentru întreaga cantitate comandată, luând în considerare ultima coală care poate fi parţial umplută. Deşeurile rezultă din diferenţa dintre suprafaţa totală a coalei şi aria ocupată de contururile aşezate; pentru ultima coală, cantitatea de deşeuri este ajustată proporţional cu numărul efectiv de contururi. Întregul tabloul, coala potrivita, număr de cutii per coală, coli necesare, deşeuri şi eficienţă procentuală este agregat într-un vector de rezultate, care apoi este sortat după cantitatea de deşeuri totale. Prima intrare oferă soluţia optimă, minimizând consumul de material.

În acest mod, functia de optimizare oferă nu doar un răspuns numeric, ci un set complet de indicatori care pot fi afișați operatorului pentru luarea deciziilor, prezentând atât opţiunea cea mai economică, cât şi detaliile geometrice şi cantitative care stau la baza ei.

6.3.1. Desfășurarea dimensiunilor

Cartonul se subțiază pe liniile de crease. Prin urmare, la fiecare latură internă a obiectului adăugăm grosimea g de două ori, astfel încât după pliere sa nu pierdem din spatiu.

```
$Lext = $lungime + 2 * $grosime;
$Wext = $latime  + 2 * $grosime;
$Hext = $inaltime + 2 * $grosime;
```

$$L_{\text{ext}} = L + 2\,g,$$
$$W_{\text{ext}} = W + 2\,g,$$
$$H_{\text{ext}} = H + 2\,g.$$

Figura 14: Calcul 6

6.3.2. Determinarea flapei de lipire

FEFCO 201 prevede o flapsă de lipire pe una dintre fețele laterale pentru suprapunerea marginilor. Dimensiunea ei crește odată cu latura maximă, în trei praguri practice.

```
$dim_max = max($Lext, $Wext);
if ($dim_max < 300)      $flapa_lipire = 20;
elseif ($dim_max < 600)  $flapa_lipire = 30;
else                      $flapa_lipire = 45;
```

$$f = \begin{cases} 20\ \text{mm}, & \max(L_{\text{ext}}, W_{\text{ext}}) < 300, \\ 30\ \text{mm}, & 300 \leq \max(L_{\text{ext}}, W_{\text{ext}}) < 600, \\ 45\ \text{mm}, & \max(L_{\text{ext}}, W_{\text{ext}}) \geq 600. \end{cases}$$

Figura 15: Calcul 7

6.3.3. Calculul clapetelor de acoperire

Clapetele superioare și inferioare acoperă părțile de sus/jos ale cutiei. Ele compensază spațiul intern și grosimea materialului. E nevoie de 2 dimensiuni diferite deoarece la inchidere, daca ele vor avea aceeasi inaltime, se vor suprapune la asamblare si vor exercita presiune una asupra celeilalte, cauzand deformari cutiei si lipsa de rigididate. Calculul este unul simplu, pentru o inchidere in linie cu standardul 201, clapetele opuse se vor intalni la mijlocul distantei laturii opuse, de acolo formula de jumatate a celei mai mici laturi dintre lungime si latime.

```
$clapa_mica = floor(min($Lext, $Wext) / 2) + $grosime;
$clapa_mare = $clapa_mica + 2;
```

$$c_{\text{mic}} = \left\lfloor \tfrac{1}{2} \min(L_{\text{ext}}, W_{\text{ext}}) \right\rfloor + g,$$
$$c_{\text{mare}} = c_{\text{mic}} + 2\ \text{mm}.$$

Figura 16: Calcul 8

6.3.4. Definirea suprafeței necesare pentru taiere

Pentru acest algoritm am decis sa tratez cutia desfasurata ca un dreptunghi, laturile fiind calculabile folosind variabile calculate anterior. Aici calculam aria acestui dreptunghi care ne va ajuta la calculul final de asezare in coala.

```
$cutie_lungime  = 2 * ($Lext + $Wext) + $flapa_lipire;
$cutie_inaltime = $Hext + 2 * $clapa_mare;
$aria_cutie     = $cutie_lungime * $cutie_inaltime;
```

$$L_{\text{contur}} = 2\,(L_{\text{ext}} + W_{\text{ext}}) + f,$$
$$H_{\text{contur}} = H_{\text{ext}} + 2\,c_{\text{mare}},$$
$$A_{\text{cutie}} = L_{\text{contur}} \times H_{\text{contur}}.$$

Figura 17: Calcul 9

6.3.5. Iterarea prin toate dimensiunile standard de coală

Întai am definit un tablou cu dimensiunile standard din hala de productie pentru materia prima, usor modificabil si usor de implementat.

```
$coli = [
        ["lungime" => 1200, "latime" => 800],
        ["lungime" => 1400, "latime" => 1000],
        ["lungime" => 1600, "latime" => 1200],
        ["lungime" => 2500, "latime" => 1250],
        ["lungime" => 2200, "latime" => 1300],
        ["lungime" => 2400, "latime" => 1400],
        ["lungime" => 3000, "latime" => 1500],
        ["lungime" => 2800, "latime" => 1600],
        ["lungime" => 3200, "latime" => 1800],
        ["lungime" => 4000, "latime" => 2100],
        ["lungime" => 3800, "latime" => 2300],
        ["lungime" => 4500, "latime" => 2500],
        ["lungime" => 5000, "latime" => 2900]
    ];
```

Figura 18: Calcul 10

După ce conturul cutiei desfasurate a fost definit (dimensiuni exterioare, flapă, clapete şi aria plană), funcţia optimize parcurge în mod sistematic lista de coli standard pentru a afla

pe care dintre ele se obţine cea mai mica risipa, deci varianta optima. Acest proces se desfăşoară într-un ciclu care, pentru fiecare intrare în tabloul de coli, realizează urmatorii pasi:

Pentru fiecare coală, preluăm dimensiunile și calculăm aria:

```
foreach ($coli as $coala) {
    $cL        = $coala['lungime'];
    $cW        = $coala['latime'];
    $aria_coala = $cL * $cW;
```

Figura 19: Calcul 11

Apoi, încercam 2 orientări diferite. Întai fără a roti coala la 90 grade, apoi cu rotire, iar la final alegem varianta unde încap cele mai multe cutii. Simplificat, verificăm de cate ori incape lungimea și lățimea cutiei desfașurate pe lungimea sau lățimea colii, în 2 variante.

```
// fără rotire
$nx   = floor($cL / $cutie_lungime);
$ny   = floor($cW / $cutie_inaltime);

// cu rotire
$nx_r = floor($cL / $cutie_inaltime);
$ny_r = floor($cW / $cutie_lungime);

// varianta cea mai buna
$max_cutii = max($nx * $ny, $nx_r * $ny_r);
if ($max_cutii === 0) {
    continue; // coală prea mica
}
```

$$n_1 = \left\lfloor \frac{c_L}{L_{\text{contur}}} \right\rfloor \times \left\lfloor \frac{c_W}{H_{\text{contur}}} \right\rfloor,$$
$$n_2 = \left\lfloor \frac{c_L}{H_{\text{contur}}} \right\rfloor \times \left\lfloor \frac{c_W}{L_{\text{contur}}} \right\rfloor,$$
$$n_{\max} = \max(n_1, n_2).$$

Figura 20: Calcul 12

In pasul urmator calculam numarul necesar de coli pentru comanda:

```
$coli_necesare = ceil($numar_cutii / $max_cutii);
```

$$N = \left\lceil \frac{q}{n_{\max}} \right\rceil.$$

Figura 21: Calcul 13

Dupa ce avem numarul de cutii pe fiecare coala si ariile cutiei si a colii, putem calcula un indice de risipa pe fiecare coala:

```
$waste_per_coala = $aria_coala - $max_cutii * $aria_cutie;
```

$$W_{\text{coală}} = (c_L \times c_W) - n_{\max} \times A_{\text{cutie}}.$$

Figura 22: Calcul 14

Urmează să calculam risipa totală de material, dar trebuie să avem grijă la următorul aspect: dacă numărul total de cutii nu se împarte exact la numărul de cutii ce încap pe coală, risipa totală trebuie ajustat în așa fel încât să reflecte realitatea cu acurateț e. Dacă de exemplu pe o coală încap 4 cutii, iar numărul total de cutii din comandă nu se divide cu 4, indicele de risipă per coala nu va reflecta situația reală pe ultima coală din proces. Astfel am făcut 2 cazuri separate:

```
$rest = $numar_cutii % $max_cutii;
if ($rest === 0) {
    $waste_total = $waste_per_coala * $coli_necesare;
} else {
    $waste_last  = $aria_coala - $rest * $aria_cutie;
    $waste_total = ($coli_necesare - 1) * $waste_per_coala + $waste_last;
}
```

Figura 23: Calcul 15

Mai departe, trecem la indicele procentual de eficienta pentru toata comanda, unde calculam raportul de material folosit efectiv pentru cutii si totalitatea materialului necesar. Acest indice e unul extrem de folositor, ajutand firma sa vizualizeze foarte usor cat de eficienta si rentabila e fiecare comanda in productie, oferind posibilitati de ajustare in dimensiunile colilor pe viitor. Calculul efectiv inmulteste aria fiecarei cutii desfasurate cu numarul de cutii obtinand suprafata totala de material ce intra in compozitia produsului finit, dupa care face raportul intre aceasta arie si aria totala de materie prima folosita, iar la final o simpla inmultire cu 100 pentru transformare in procent.

```
$eficienta_pct = ($aria_cutie * $numar_cutii) / ($aria_coala * $coli_necesare) * 100;
```

Figura 24: Calcul 16

În final, am adaugat fiecare indice relevant într-un tablou de rezultate, pe care îl voi sorta după cea mai mică risipă, astfel afișând soluția optim în interfață, plus indici interesanți și utili.

```
$rezultate[] = [
                "coala" => "{$cL}x{$cW}",
                "cutii_per_coala" => $max_cutii,
                "aria_cutie" => $aria_cutie,
                "aria_coala" => $aria_coala,
                "coli_necesare" => $coli_necesare,
                "waste_per_coala" => $waste_per_coala,
                "waste_total" => $waste_total,
                "eficienta"   => $eficienta_pct

            ];
```

```
usort($rezultate, fn($a, $b) => $a['waste_total'] <=> $b['waste_total']);

return $rezultate[0] ?? null;
```

Figura 25: Calcul 17

6.4. Generarea proiect taiere pentru productie

Pentru a transforma desenul planificat de nesting într-un fișier gata de trimis către utilajele CNC, am implementat un generator de SVG, bazat pe modelul FEFCO 201. Acest modul construiește, într-un singur pas, conturul complet al cutiei desfășurate, care poate fi vazut apoi in interfata business. Modelul FEFCO 201 a fost respectat pentru construirea desenelor vectoriale conforme cu standardul industriei (FEFCO-European Federation of Corrugated Board Manufacturers, 2018).

Modulul de generare a fișierelor SVG reprezintă legătura directă dintre planul de nesting și mașinile de taiere industriale. Într-o singură funcție PHP, construim un desen vectorial care include toate elementele unei cutii FEFCO 201 desfășurate: cele patru panouri laterale, flapa de lipire și clapetele superioare și inferioare. Alegerea formatului SVG cu unități în milimetri asigură că utilajele CNC vor interpreta direct dimensiunile, eliminând orice conversie manuală și reducând timpul de set-up la minimum.

Primul pas este inițializarea „canvas-ului” SVG: calculăm lățimea totală drept suma lățimii flapei de lipire și a celor patru panouri, iar înălțimea totală ca suma celor două clapete și a înălțimii exterioare. Apoi, antetul vectorial este generat dinamic, cu atributele “width” și “height” în milimetri și un viewBox corespunzător, pentru a garanta redarea la scară reală.

Acest procedeu simplu, dar foarte robust, permite ca la fiecare apel (de exemplu, când un operator dă click pe butonul „ SVG” din interfața de administrare) să fie creat instant un fișier cu numele cutie_LxWxH.svg, gata de descărcare sau redirecționare către utilaj.

În continuare, desenăm întâi flapa de lipire sub forma unui poligon trapezoidal, poziționat automat în stânga primului panou. Acest element asigură zona de suprapunere și aplicarea adezivului, iar offset-ul vertical regulat în funcție de dimensiunile cutiei (10, 20 sau 25 mm) menține estetica și funcționalitatea geometrică. După flapă, bucla care parcurge array-ul cu lățimile panourilor inserează cele patru dreptunghiuri principale, aliniindu-le pe orizontală. În final, pentru fiecare panou adăugăm două clapete - un dreptunghi deasupra și unul dedesubt - care închid fețele superioare și inferioare ale structurii, furnizând un suport fizic complet la îndoire.

Modulul de generare SVG este gândit ca ultima verigă a lanțului nostru: după ce am primit dimensiunile interne ale cutiei și le-am optimizat, următorul pas este să transformăm conturul plan într-un fișier vectorial standard, gata de trimis utilajelor CNC. Ideea de bază a fost să păstrăm totul într-o singură funcție PHP, fără dependențe externe, astfel încât să putem genera rapid, în milimetri, forma FEFCO 201 desfășurată. În primul rând calculăm „canvas”-ul SVG:

```
$latime_totala = $flap_width + $latime_panouri;
$inaltime_totala = $flapa_h + $Hext + $flapa_h;
```

Figura 26: calcul 18

Apoi inițializăm documentul SVG cu un antet ce specifică width, height și viewBox în unități milimetrice, pentru a garanta redarea la scară reală. Urmează desenarea flapsei de lipire, realizată ca un element <polygon> trapezoidal, apoi bucla care plasează cele patru panouri laterale dreptunghiuri <rect> în șir, iar la final două bucle asemanătoare pentru „clapetele” superioare și inferioare. De exemplu, pentru panouri iterăm:

```
foreach ($panouri as $latime_panou) {
    $svg .= "<rect x='{$x}' y='{$y_corp}' width='{$latime_panou}' height='{$Hext}' />\n";
    $x += $latime_panou;
}
```

Iar pentru clapete:

```
$svg .= "<rect x='".($x_panou+$g/2)."' y='".($y_corp-$flapa_h)."' … />";
$svg .= "<rect x='".($x_panou+$g/2)."' y='".($y_corp+$Hext)."' … />";
```

Figura 27: Calcul 19

Crearea fișierelor SVG este faza finală a fluxului nostru de lucru automatizat pentru fabricarea ambalajelor, integrând rezultatele calculelor încastrate și convertindu-le într-o desen vectorial cuprinzător, pregătit pentru tăierea cu mașini CNC. Alegem formatul SVG în mod special pentru simplitatea sa: este un fișier text XML care delimitează forme geometrice folosind etichete precum <rect> și <polygon>, cu toate coordonatele și dimensiunile specificate în milimetri, eliminând astfel necesitatea conversiei manuale sau a software-ului intermediar. Generatorul este complet compus în PHP, fără biblioteci suplimentare, permițând instalarea și executarea pe orice server care suportă acest limbaj, asigurând astfel că codul rămâne concis și ușor de gestionat. Funcția principală începe cu dimensiunile interioare ale cutiei și grosimea cartonului, calculează ulterior zonele de pliere și suprapunere, configurează antetul SVG cu lățime, înălțime și viewBox, și apoi încorporează secvențial aripa de lipit, cele patru panouri laterale și aripile de sus și de jos, folosind bucle explicite pentru construcția elementelor vectoriale. În cele din urmă, fișierul este salvat și redirecționat către utilizator, facilitând descărcarea imediată a layout-ului extins ca document .svg. Astfel, am integrat cu succes backend-ul, care include logica de optimizare și manipularea datelor, direct în fluxul de lucru de fabricație, oferind o soluție cuprinzătoare, prietenoasă cu utilizatorul și scalabilă pentru standardele sau tipurile de ambalaje viitoare.

7. Testarea aplicatiei

Testarea servește ca mecanism de asigurare a calității pentru produsele software, completând dezvoltarea printr-un proces de verificare și validare care începe cu cerințele și se extinde la aplicarea practică în condiții reale. Testele aplicației sunt structurate conform principiilor din standardele internaționale ISO privind verificarea software (International Organization for Standardization, 2013). Obiectivul principal al testării este de a verifica dacă fiecare funcționalitate respectă specificațiile și de a identifica timpii de răspuns, comportamentele neprevăzute sau deficiențele înainte de livrarea produsului către utilizatorul final. O strategie cuprinzătoare care integrează testarea manuală și automată garantează o evaluare amănunțită a interfeței și experienței utilizatorului, alături de o validare rapidă și reproductibilă a componentelor esențiale, inclusiv algoritmii de calcul și crearea fișierelor SVG. Această abordare eradichează erorile logice, diminuează costurile corecțiilor tardive și sporește încrederea în stabilitatea și reziliența sistemului.

Un prim beneficiu al testării manuale este capacitatea de a surprinde acele defecte care, din natura lor, nu pot fi anticipate de scripturile automate: inconsistențe vizuale, mesaje de

eroare neintuitive sau scenarii de utilizare atipice. Testarea manuală simulează fidel modul în care utilizatorii interacționează cu aplicația, de la completarea configuratorului și încărcarea imaginilor, până la navigarea în panoul de administrare și descărcarea fișierelor SVG. În paralel, testele automate asigură reproducerea rapidă a seturilor de date de intrare și compararea rezultatelor cu valori de referință, oferind o acoperire extinsă și un feedback imediat la fiecare modificare de cod. Astfel, se formează un echilibru optim între flexibilitate și eficiență, permițând echipei să identifice și să remedieze rapid defectele regresive.

În proiectul nostru, această abordare mixtă a fost aplicată concret prin trei categorii de testare. Mai întâi, am verificat manual corectitudinea calculelor de nesting comparând output-ul funcției dedicate cu măsurători realizate pe hârtie. Apoi, am validat vizual fișierele SVG în programe CAD, confirmând că panourile, flapa și clapetele respectă rigorile geometrice. În final, am executat câteva tăieri de probă pe utilajul CNC, pentru a confirma că traseele generate sunt interpretate corect de mașină și că producția fizică corespunde planului digital. Prin aceste etape, testarea nu a rămas un simplu exercițiu academic, ci a devenit un instrument esențial pentru livrarea unei soluții integrate, fiabile și gata de utilizare în cadrul proceselor reale de fabricație.

Integrarea serviciului de estimare automata a dimensiunilor obiectelor, partea de testare a fost cruciala. Am observat că precizia rezultatelor crește semnificativ atunci când clientul plasează lângă obiect un element de referință cu dimensiuni cunoscute, de pildă o monedă, o riglă sau un pix standard. În acest fel, modelul AI poate calibra mai exact factorul de scalare, reducând abaterile la câțiva milimetri și evitând erorile de proporție care apar în imaginile fără referințe. Pentru confort și flexibilitate, estimarea automată este opțională, iar dacă răspunsul AI nu este suficient de satisfăcător sau dacă serviciul extern întâmpină un incident, utilizatorul poate introduce manual valorile. În plus, fiecare apel către API-ul de estimare costă sub 0,01 USD, ceea ce menține soluția eficientă și accesibilă chiar și în cazul unui volum mare de comenzi. Astfel, îmbinăm agilitatea inteligenței artificiale cu controlul manual al utilizatorului, asigurând un echilibru optim între viteză, precizie și costuri. In tabelul urmator voi ilustra cateva din cele mai importante testari facute impreuna cu statusul lor:

Sarcina	Stare
1. Autentificare	Succes
2. Comanda doar utilizatorii autentificati	Succes
3. Minimizarea risipei corect	Succes
4. Generare adecvata de schita SVG	Succes
5. Estimare precisa a dimensiunilor obiectului	Partial succes

Tabel 3: Sarcini de testare

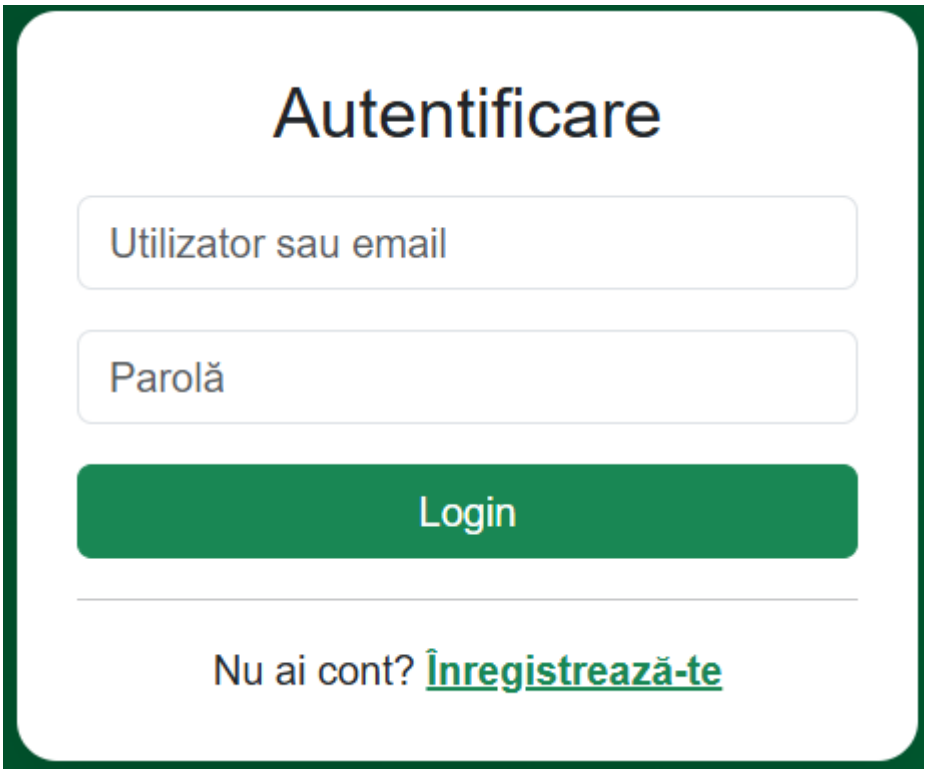


Figura 28: Autentificare

1. Pasul de autentificare se face prin pagina de Login, iar după o autentificare reușită utilizatorul vă fi redirecționat pe pagina de pornire. Autentificarea poate fi de 2 tipuri: administartor sau client.

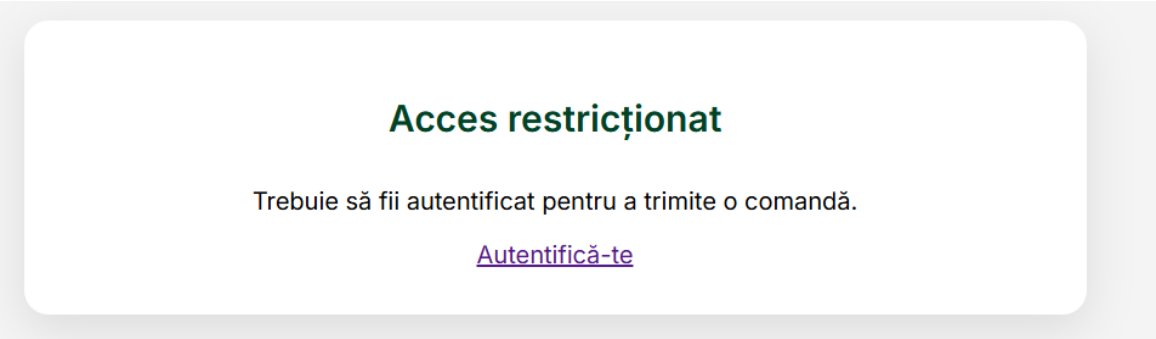
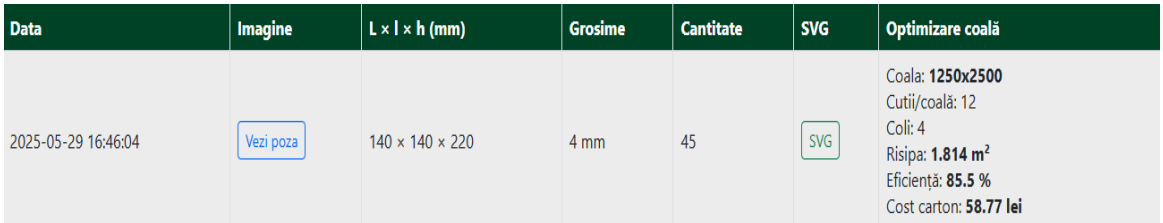


Figura 29: Autentificare 2

2. În cazul în care utilizatorul nu este autentificat, va primi un mesaj scurt în care este informat de nevoia de autentficare înainte de a configura o comanda.



Data	Imagine	L × l × h (mm)	Grosime	Cantitate	SVG	Optimizare coală
2025-05-29 16:46:04	Vezi poza	140 × 140 × 220	4 mm	45	SVG	Coala: **1250x2500** Cutii/coală: 12 Coli: 4 Risipa: **1.814 m²** Eficiență: **85.5 %** Cost carton: **58.77 lei**

Figura 30: Detalii comandă

3. Pentru testarea corectitudinii algoritmului de optimizare a risipei, am luat exemplu o comanda creata si am facut calculele necesare manual. Pentru comanda de 45 cutii cu dimensiuni 140×140×220 mm şi grosime 4 mm, algoritmul extinde mai întâi marginile cu grosimea materialului, apoi construiește conturul plan (612×386 mm) cu flapsă de 20 mm şi clapete de 78/80 mm. Pe coală 1250×2500 mm încap 12 contururi indiferent de orientare, ceea ce duce la 4 coli necesare. Suprafața totală achiziționată (12,5 m²) minus cea folosită pentru cele 45 de cutii (≈10,63 m²) generează ≈1,87 m² de deșeu, iar eficiența de material este de aproximativ 85 %. Aceste rezultate se potrivesc perfect cu valorile afișate în aplicație, confirmând corectitudinea implementării.

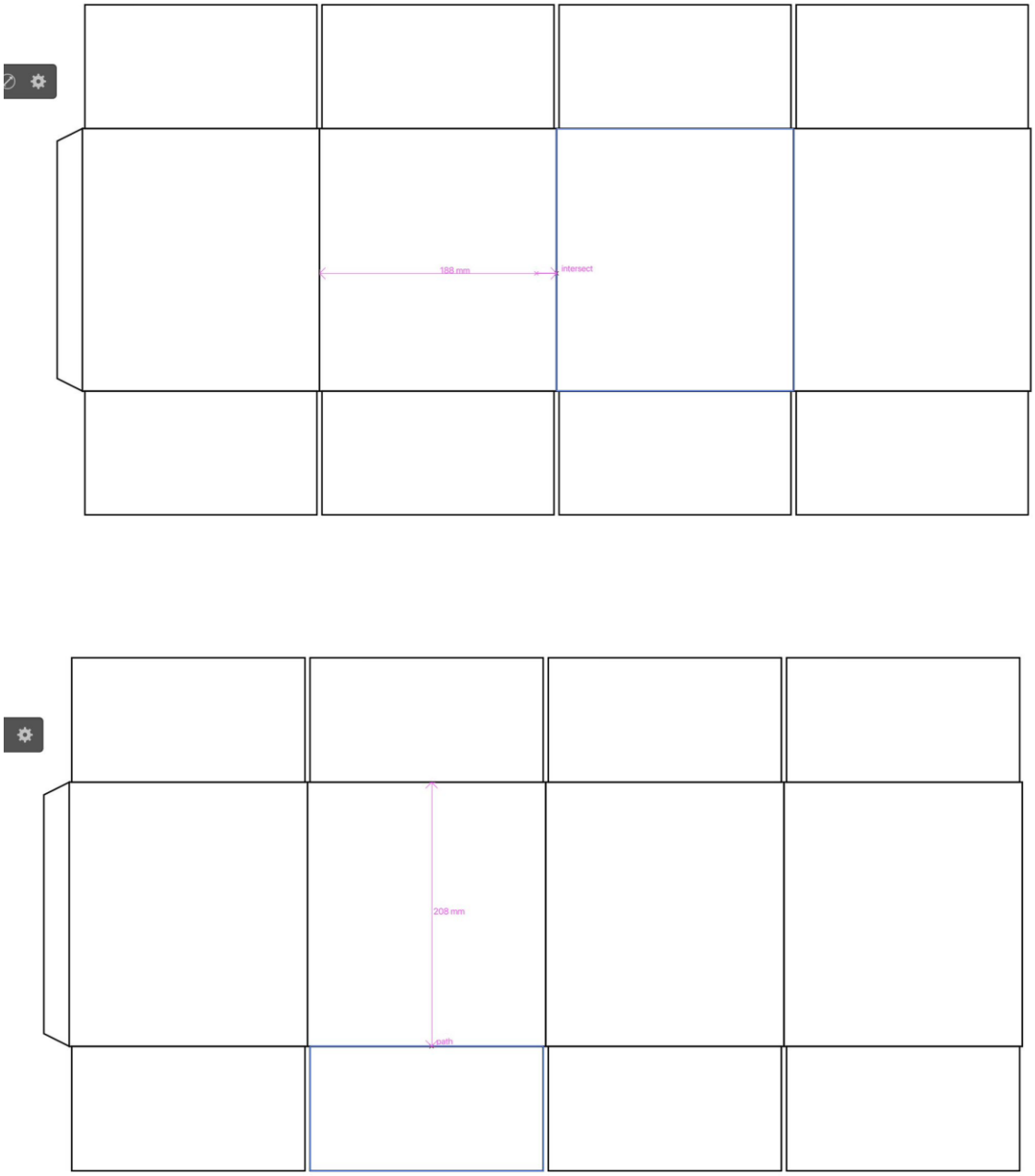


Figura 31: Verificarea integrității schiței

4. Generarea de fișier SVG a fost verificata în instrumentele cele mai folosite din industrie pentru a fi siguri că proiectul a ieșit ce ne-am dorit. Pentru acest exemolu am luat o comanda ce necesita o cutie cu dimensiunile interioare 180x180x200, cu grosimea cartonului de 4mm. Am deschis schiță rezultata în Adobe Illustrator și am verificat

dimensiunile, după care am trimis schiță către un utilaj CNC pentru taiere, unde am obținut exact cutia dorita.

5. Estimarea dimensiunilor prin intermediul serviciului OpenAI API s-a dovedit a fi folositoare la nivel de client, facilitând procesul de comanda pentru clienții care nu cunosc exact dimensiunile obiectului lor, sau pentru cei care vor doar să își facă o idee cât i-ar costa ambalajul pentru un produs. Cu toate acestea, procesul s-a dovedit a fi uneori mai puțin precis, mai exact atunci când obiectul central din imagine nu este însoțit de un obiect de referință suficient de comun. În consecință, am adăugat un scurt mesaj în care clientul este încurajat să includă în imagine un obiect de referință pentru rezultate mai bune. Per total, serviciul de estimare automata al configuratorului este unul folositor pentru clienții noi sau pentru cei care vor să facă o mica simulare de costuri.

Testarea întregii aplicații ne-a demonstrat că fiecare etapă, de la introducerea comenzilor și optimizarea nesting-ului, până la generarea desenelor SVG și afișarea lor în interfața de administrare, funcționează așa cum ne-am propus. Prin combinația dintre verificări manuale, care au surprins scenarii neașteptate și au validat experiența utilizatorului, și teste automate, care au confirmat constantă corectitudinea calculelor și integritatea fișierelor, am obținut un produs matur și fiabil. În plus, încercările efectuate direct pe utilajele CNC au încheiat ciclul de verificare, asigurându-ne că rezultatele digitale se traduc în realitate exact așa cum a fost proiectat. Astfel, aplicația indeplineste cerintele propuse, lasand loc si de imbunatari pe viitor pentru a trece la urmatorul nivel de implementare reala in contextul industriei de ambalare.

Concluzie și direcții viitoare

Scopul proiectului actual a fost să rezolve o problemă cu care se confruntă sectorul ambalajelor din carton: absența unui sistem digital centralizat și eficient care să permită configurarea, estimarea, optimizarea și expedierea comenzilor într-un mod automatizat și ușor accesibil. Scopul nostru în crearea aplicației web Kaydara Box Configurator a fost să oferim o soluție utilă, scalabilă și prietenoasă cu utilizatorul, care să asiste atât producătorii de ambalaje, cât și utilizatorii finali.

Dificultățile în comunicarea eficientă a nevoilor, lipsa automatizării în calculul dimensiunilor și optimizarea slabă a consumului de materiale sunt doar câteva dintre problemele actuale ale procesului tradițional de comandă pe care le-am evidențiat în această lucrare. Aceste restricții duc frecvent la eforturi irosite, execuție slabă, deșeuri de carton și o experiență neplăcută pentru clienți. Remediul sugerat este o reacție concretă la aceste probleme.

Aplicația creată are o serie de elemente de vârf:

Un API de inteligență artificială este folosit pentru a estima automat dimensiunile pe baza unei imagini încărcate de client; utilizatorii care cunosc dimensiunile precise pe care le doresc pot configura manual; un algoritm proprietar de nesting optimizează foaia de carton, minimizând deșeurile și crescând eficiența producției; fișierele SVG sunt generate automat, fiind compatibile cu mașinile CNC utilizate în producție (cum ar fi Zünd), eliminând necesitatea unui designer grafic în lanțul de procesare; și interfața este prietenoasă și intuitivă atât pentru client, cât și pentru echipa de afaceri.

Inspirată de tehnica Agile și modificată pentru o lucrare de licență, aplicația a fost dezvoltată folosind un model iterativ-incremental. Utilizatori reali și reprezentanți ai unei industrii de ambalare din apropiere au colaborat pentru a planifica, testa și valida fiecare etapă de dezvoltare. Feedback-ul a fost integrat continuu, iar cerințele operaționale au fost ajustate rapid datorită contactului constant cu utilizatorii. Ca urmare, proiectul a progresat constant, dar sub control, producând o aplicație utilă care este simplu de întreținut și de extins.

Tehnic, a fost selectată o arhitectură simplă, dar fiabilă: MySQL și fișiere CSV pentru stocarea hibridă a datelor, HTML/CSS/JavaScript pentru interfața utilizatorului, Bootstrap pentru designul receptiv și un backend PHP "vanilla" pentru accesibilitate și ușurință în întreținere. Soluția selectată asigură simplitatea adoptării, costuri reduse de implementare și compatibilitate cu infrastructura actuală a afacerilor.

Din punct de vedere al impactului, aplicația sugerată:

- Reduce timpul de execuție și simplifică procesul de comandă;
- Minimizează eroarea umană în procesarea cerințelor;
- Oferă afacerii date standardizate, ușor de înțeles și gata pentru producție;
- Optimizează consumul de carton și ajută la asigurarea sustenabilității procesului de fabricație;
- Oferă o experiență digitalizată contemporană, ușor de utilizat chiar și de utilizatori non-tehnici.

Cu potențialul de a încorpora module suplimentare, cum ar fi plățile online, urmărirea comenzilor, integrarea cu ERP-urile industriale sau exporturile automate către inventarele de materii prime, această soluție are potențialul de a se dezvolta într-o platformă cuprinzătoare pentru configurarea și gestionarea comenzilor în sectorul ambalajelor în timp. În plus, programul poate servi ca un model care poate fi replicat în alte afaceri cu cerințe comparabile, cum ar fi tipografiile, produsele promoționale și decorațiunile personalizate.

Există mai multe modalități prin care soluția poate fi dezvoltată și îmbunătățită, chiar dacă aplicația existentă oferă o bază solidă și o gamă completă de funcționalități pentru prototipul de demonstrație:

- Conectivitate la o platformă de plată pe internet

Prin includerea unei funcții de plată cu cardul sau a altor opțiuni de plată digitală (cum ar fi Stripe sau PayPal), comenzile ar putea fi îndeplinite complet fără a necesita implicarea umană, sporind eficiența și încrederea procesului.

• Un modul pentru urmărirea comenzilor

O experiență asemănătoare sistemelor contemporane de comerț electronic ar fi oferită printr-o funcție care permite clientului să vizualizeze starea comenzii în timp real (plasată, în procesare, finalizată, livrată).

• Imbunatatirea estimarii bazate pe inteligența artificială

Estimarea dimensiunilor din configurator ar putea fi imbunatatita in timp cu un model mai bun de recunoastere, fie antrenat de noi, fie un serviciu extern mai capabil.

• Adaptabilitate crescută la diferite tipuri de ambalaje

Aplicația este în prezent adaptată pentru modelele de tip FEFCO 201. Aplicarea platformei ar fi crescută prin extinderea la forme și tipuri suplimentare de cutii (cum ar fi cutiile cilindrice și cutiile cu capac).

• Dezvoltarea unei aplicații mobile

Comenzile de teren pot fi configurate rapid prin conversia platformei într-o aplicație Android sau iOS, ceea ce ar fi util pentru distribuitori sau clienți business-to-business.

• Integrarea sistemului ERP

Recepția comenzilor, controlul stocurilor și facturarea sunt doar câteva dintre procesele interne care ar putea fi complet automatizate prin conectarea aplicației la un ERP existent (cum ar fi SAP, Microsoft Dynamics sau Odoo).

• O interfață administrativă sofisticată

Managerii ar putea lua decizii mai bune privind comenzile, pierderile, programul de livrări și alte subiecte dacă s-ar adăuga un tablou de bord cu statistici cuprinzătoare, grafice interactive și rapoarte exportabile.

• Traduceri și internaționalizare

Ar fi logic să traducem interfața aplicației în alte limbi și să o adaptăm la diferite monede și unități de măsură pentru a ne extinde internațional.

• Automatizare completă a livrării

Integrarea cu furnizorii de curierat (cum ar fi Fan Courier, GLS și DPD) pentru a genera AWB-uri și a trimite notificări de expediere automat.

• Implementarea unei soluții de autentificare multifactorială

Ar fi avantajos să folosim email temporar, aplicații mobile sau autentificare prin SMS pentru o securitate sporită, în special în secțiunea de administrare.

Lucrarea de licență se încheie prin prezentarea modului în care tehnologiile web și inteligența artificială pot fi utilizate pentru a digitaliza cu succes un domeniu industrial convențional. Aplicatia Kaydara Box Configurator este un pas important spre automatizarea, eficiența și profesionalizarea procesului de configurare și producție a ambalajelor din carton, deoarece adoptă o abordare pragmatică centrată pe cerințele reale ale clienților. Proiectul prezintă o imagine realistă a viitorului digital al sectorului de ambalaje personalizate, pe lângă un prototip funcțional.

Bibliografie

Boston Consulting Group. (2012). *The automation advantage in manufacturing.* BCG: Boston.

Codd, E. (1970). A Relational Model of Data for Large Shared Data Banks. *Communications of the ACM*.

FEFCO-European Federation of Corrugated Board Manufacturers. (2018). *FEFCO Code-International Packaging Design Standards.* Brussels: FEFCO.

Group, P. (2024). *PHP Manual-Password Hashing*. Retrieved from https://www.php.net/manual/en/function.password-hash.php

International Organization for Standardization. (2013). *ISO/IEC/IEEE 29119 - Software and Systems engineering - Software testing.* Geneva: ISO.

Li, Y. a. (2015). A 2D Nesting Algorithm for Cutiing Optimization. *Journal of Manufacturing Systems*, 110-119.

OpenAI. (2024). *OpenAI API Documentation-GPT-4o.* San Francisco: OpenAi.

Robertson, G. L. (2012). *Food Packaging: Principles and Practice.* CRC Press.

Sommervile, I. (2015). *Software Engineering.* London: Pearson Education.

Soroka, W. (2002). *Fundamentals of Packaging Technology.* Naperville: Institute of Packaging Professionals.